

Panduan Simbol, Angka, Terminal & Logika Coding

PowerShell, CMD, winget, Python, JavaScript, JSON, Git, warna kode, membaca log, dan cara berpikir seperti programmer

Versi 2. Dari nol: arti tiap simbol, arti angka, cara membuka dan membedakan terminal, arti warna, cara membaca pesan error dan log, contoh kode jadi yang sudah diuji, dan sistem logika di balik semuanya.

Daftar isi

1. Peta besar: apa yang perlu dipahami dan apa yang wajib dihafal
 2. Warna: cara membaca kode dan terminal dari warnanya (benar atau salah)
 3. Terminal: jenis, cara membuka, hitam atau biru, perbedaan antar terminal, PATH, admin
 4. winget dan pemasang paket lain
 5. Simbol PowerShell
 6. Python
 7. Arti angka
 8. JSON, .env, JavaScript, HTML, Bash
 9. Git
 10. Sistem logika
 11. Membaca error dan log (termasuk log MongoDB)
 12. Bedah perintah KelasTrack
 13. Cheat sheet
- Lampiran: semua simbol keyboard, simbol gabungan, regex, URL, shortcut VS Code

Cara pakai

Contoh kode di buku ini sudah diwarnai seperti editor VS Code tema terang. Lihat bab 2 untuk arti tiap warna.

Kotak **hijau** = tips. Kotak **kuning** = hati-hati.

Contoh Python, JavaScript, JSON dan Bash sudah dijalankan/diperiksa saat membuat panduan ini. Contoh PowerShell dan winget belum bisa dijalankan di lingkungan pembuat (bukan Windows), jadi coba sendiri dan baca pesan error-nya dengan bab 11.

1. Peta besar: apa yang perlu dipahami dan dihafal

Coding itu sebenarnya tiga hal yang bekerja bersama. Kalau bingung, tanya diri sendiri: masalah gua ada di bagian yang mana?

Bagian	Isinya	Contoh masalah
1. Bahasa & simbol	Aturan penulisan: kata kunci, tanda baca, tanda kutip, kurung, angka.	<code>SyntaxError</code> , lupa titik dua, kutip tidak berpasangan.
2. Alat	Terminal, editor (VS Code), Git, pemasang paket (pip, npm, winget), virtual environment.	<code>not recognized</code> , <code>ModuleNotFoundError</code> , salah folder.
3. Cara berpikir (logika)	Urutan, pilihan, perulangan, fungsi, data; memecah masalah; mencari bug.	Hasil salah padahal tidak ada error, loop tak berhenti.

Kabar baiknya: bagian 1 dan 2 bisa dilihat ulang kapan saja (tidak perlu hafal semuanya). Bagian 3 yang paling berharga karena berlaku di semua bahasa.

1.1 Tiga tingkat: hafal, paham, cukup tahu

Tingkat	Artinya	Isinya
A. Hafal di luar kepala	Dipakai tiap hari. Harus bisa diketik tanpa melihat. (Daftar di 1.2)	sekitar 60 item
B. Paham konsep, cari sintaks saat perlu	Tahu fungsinya dan kapan dipakai; detail penulisan boleh buka cheat sheet.	Operator PowerShell, regex, opsi <code>git</code> , opsi <code>winget</code> , modul Python.
C. Cukup tahu ada	Cukup ingat 'ini pernah ada' lalu cari di Google/dokumentasi.	Ratusan cmdlet, ribuan library, kode error langka.

1.2 Daftar wajib hafal (tingkat A)

Area	Yang dihafal	Jumlah
Terminal (semua)	<code>cd</code> , <code>cd ..</code> , <code>pwd</code> , <code>ls/dir</code> , <code>mkdir</code> , <code>cat/type</code> , <code>cp</code> , <code>mv</code> , <code>rm</code> , <code>clear/cls</code> ; tombol Tab , Ctrl+C , panah atas; <code>.</code> = folder ini; kutip untuk spasi.	13
PowerShell	<code>Get-ChildItem</code> , <code>Set-Location</code> , <code>Get-Content</code> , <code>Copy-Item</code> , <code>Remove-Item</code> , <code>Test-Path</code> , <code>Write-Host</code> , <code>Get-Command</code> , <code>Get-Help</code> , <code>\$variabel</code> , pipa <code> </code> , <code>\$_</code> , <code>.\script.ps1</code> .	13
Operator & logika	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code> ; <code>and</code> <code>or</code> <code>not</code> ; PowerShell <code>-eq</code> <code>-ne</code> <code>-gt</code> <code>-lt</code> <code>-and</code> <code>-or</code> <code>-not</code> ; <code>%</code> sisa bagi; <code>=</code> mengisi vs <code>==</code> membandingkan.	12
Python inti	<code>print</code> , <code>input</code> , <code>len</code> , <code>range</code> , <code>int/str/float</code> , <code>if/elif/else</code> , <code>for</code> , <code>while</code> , <code>def+return</code> , list <code>[]</code> , dict <code>{}</code> , <code>import</code> , <code>try/except</code> , f-string, indentasi + titik dua.	16
Python alat	<code>python --version</code> , <code>python -m venv .venv</code> , aktifkan venv, <code>pip install</code> , <code>pip install -r requirements.txt</code> , <code>deactivate</code> .	6
Git	<code>git status</code> , <code>git add .</code> , <code>git commit -m "..."</code> , <code>git push</code> , <code>git pull</code> , <code>git clone</code> , <code>git checkout -b</code> .	7
winget	<code>winget search</code> , <code>winget install --id X -e</code> , <code>winget list</code> , <code>winget upgrade</code> , <code>winget uninstall</code> , <code>winget show</code> .	6
Angka & konsep	Indeks mulai 0; exit code 0 = sukses; <code>localhost</code> = komputer sendiri; port; HTTP 200/404/500; versi <code>mayor.minor.patch</code> .	6

Cara menghafal yang efektif: jangan menghafal dari daftar. Ketik perintahnya tiap hari di proyek kecil; dalam seminggu jari akan hafal sendiri. Yang lupa, cari di bab cheat sheet (bab 13).

1.3 Konsep inti (tingkat B): satu kalimat + perumpamaan

Konsep	Artinya	Perumpamaan
Variabel	Nama yang menyimpan sebuah nilai: <code>umur = 16</code> .	Kotak berlabel.
Tipe data	Jenis nilai: angka, teks, benar/salah, daftar, kamus.	Jenis barang di kotak: angka tidak bisa dijumlah dengan kata.
Fungsi	Potongan kode bernama yang menerima input dan memberi hasil.	Mesin: masuk bahan, keluar produk.
Kondisi (if)	Memilih jalur berdasarkan benar/salah.	Persimpangan jalan.
Perulangan (loop)	Mengulang langkah untuk tiap item atau sampai syarat tercapai.	Menyetempel tiap lembar dalam tumpukan.
List / dict	Daftar berurutan / pasangan kunci-nilai.	Daftar belanja / buku telepon.
Path (jalur file)	Alamat file. Mutlak (<code>C:\a\b.txt</code>) atau relatif (<code>.\b.txt</code>).	Alamat rumah lengkap vs 'rumah sebelah'.
PATH (huruf besar)	Daftar folder yang dicari terminal saat lu mengetik nama program.	Daftar toko tempat terminal mencari barang.
Environment variable	Variabel pengaturan di luar kode (<code>\$env:Path, .env</code>).	Papan pengumuman yang bisa dibaca semua program.
Proses	Program yang sedang berjalan. <code>Ctrl+C</code> menghentikannya.	Karyawan yang sedang bekerja.
stdin / stdout / stderr	Aliran masuk, aliran keluar normal, aliran keluar untuk error.	Pipa air masuk, pipa air bersih, pipa air kotor.
Client - Server	Client (browser) meminta, server menjawab.	Pelanggan dan pelayan restoran.
API / request / response	Alamat yang bisa dipanggil untuk meminta atau mengubah data.	Menu restoran: kode pesanan dan jawabannya.
Database & CRUD	Tempat data tersimpan. Create, Read, Update, Delete.	Lemari arsip.
JSON	Format teks untuk menukar data antar program.	Formulir isian standar.
Dependency / package	Library pihak lain yang dipakai proyek.	Alat pinjaman dari toko perkakas.
Virtual environment	Kotak terpisah berisi library satu proyek.	Kotak perkakas tiap proyek.
Commit / branch	Titik simpan versi / jalur kerja terpisah.	Save point di game / draft alternatif.
Log	Catatan otomatis apa yang dilakukan program.	Buku harian program.
Dev vs Production	Mode pengembangan (aman diutak-atik) vs mode asli dipakai pengguna.	Latihan vs pertunjukan.

1.4 Urutan belajar yang disarankan

- **Minggu 1:** terminal (bab 3) + simbol dasar (bab 5, 7). Target: bisa pindah folder dan menjalankan script tanpa bingung.
- **Minggu 2:** Python dasar (bab 6) + logika (bab 10). Target: bisa menulis program 20 baris sendiri.
- **Minggu 3:** Git (bab 9), pip/venv, winget (bab 4). Target: menyimpan proyek ke GitHub.
- **Minggu 4:** JSON/API (bab 8), membaca error & log (bab 11). Target: bisa debug sendiri.
- **Setiap hari:** 15 menit mengetik ulang contoh, lalu sengaja merusaknya dan membaca error-nya.

2. Warna: cara membaca kode dan terminal dari warnanya

Warna adalah alat cek tercepat. Editor (VS Code, dll) mewarnai tiap bagian kode sesuai perannya (**syntax highlighting**). Kalau lu menetik dengan benar, warnanya 'masuk akal'. Kalau ada yang salah ketik, warnanya biasanya **tiba-tiba berubah atau hilang**. Terminal juga memakai warna dengan aturan yang hampir sama di mana-mana.

2.1 Peta warna kode (sama dengan warna di semua contoh buku ini)

Peran	Tema terang (buku ini)	Tema gelap (Dark+)	Kalau salah, tandanya
Kata kunci (<code>def</code> , <code>class</code> , <code>function</code> , <code>const</code>)	<code>def class const function</code>	<code>def class const function</code>	Tidak berwarna / sama dengan teks biasa -> salah ketik (<code>Def</code> , <code>fucntion</code>) atau file belum dikenali bahasanya.
Kontrol alur (<code>if</code> , <code>for</code> , <code>return</code> , <code>import</code> , <code>-eq</code>)	<code>if for return import -eq</code>	<code>if for return import -eq</code>	Tidak ungu -> salah ketik atau salah huruf besar-kecil.
Teks dalam kutip (string)	<code>"halo dunia"</code>	<code>"halo dunia"</code>	Warna string meluas ke sisa baris/file -> ada kutip yang belum ditutup.
Angka	<code>42 3.14 0xFF</code>	<code>42 3.14 0xFF</code>	Angka tidak berwarna hijau -> ada kutip di sekitarnya (jadi teks) atau salah ketik (<code>40</code> , huruf <code>O</code>).
Komentar (tidak dijalankan)	<code># catatan</code>	<code># catatan</code>	Semua kode berikutnya jadi hijau -> ada <code>/*</code> , <code><#</code> , atau <code>"""</code> yang belum ditutup.
Nama fungsi / perintah	<code>print()</code> <code>Get-ChildItem</code>	<code>print()</code> <code>Get-ChildItem</code>	Tidak berwarna -> fungsi belum didefinisikan / belum di-import / salah ketik.
Variabel & properti	<code>\$nama umur self.x</code>	<code>\$nama umur self.x</code>	Variabel baru yang tak pernah dipakai bisa tampak redup.
Class / tipe data	<code>Siswa int List</code>	<code>Siswa int List</code>	Nama class salah huruf besar-kecil -> warna lain.
Parameter PowerShell	<code>-Recurse -Force</code>	<code>-Recurse -Force</code>	Parameter salah ketik -> tidak berwarna / garis merah.
Tag HTML dan atribut	<code><div> class="x"</code>	<code><div> class="x"</code>	Tag tidak ditutup -> warna berantakan di bawahnya.

Warna persisnya bisa beda sedikit antar tema dan editor, tapi **peran warnanya sama**: kata kunci biru/ungu, teks merah-oranye, komentar hijau, angka hijau muda. Kalau lu pakai tema gelap, pakai kolom 'Dark+'.

Contoh: kode benar vs kode yang salah kutip

```
nama = "Budi"           # benar: "Budi" berwarna string, komentar hijau
print("Halo " + nama)
```

Sekarang lupa menutup kutip pada baris pertama: `nama = "Budi`. Di editor, **seluruh sisa baris (termasuk komentarnya) berubah jadi warna string**, lalu ada garis bergelombang merah di bawahnya. Saat melihat warna string 'bocor' ke mana-mana, cari kutip yang hilang.

```
nama = "Budi           # <- salah: kutip belum ditutup, sisa baris ikut jadi warna string
print("Halo " + nama)
```

2.2 Penanda lain di editor (VS Code)

Yang terlihat	Artinya	Tindakan
Garis bergelombang merah	Error: kode tidak akan jalan.	Arahkan mouse ke garis untuk membaca penyebab; buka panel Problems (Ctrl+Shift+M).
Garis bergelombang kuning	Peringatan: jalan, tapi mencurigakan.	Baca dulu; biasanya variabel tak terpakai atau gaya penulisan.
Garis bergelombang biru/abu	Saran / informasi.	Boleh diabaikan.
Teks redup/abu pada variabel atau import	Dideklarasikan tapi tidak dipakai.	Hapus atau pakai.
Kurung berwarna-warni berpasangan	Pasangan () [] { } punya warna sama.	Kurung tanpa pasangan tampil merah: ada yang kurang/lebih.
Pojok kanan bawah tertulis Plain Text	Editor tidak tahu bahasanya, jadi tidak ada warna sintaks.	Klik tulisan itu lalu pilih Python / PowerShell / dll, atau simpan file dengan ekstensi benar (.py, .ps1).
Tab file berpoin putih/bulat di judul	Ada perubahan belum disimpan.	Ctrl+S.
Nama file hijau + huruf U	Untracked: file baru yang belum dilacak Git.	<code>git add</code> .
Nama file kuning/oranye + M	Modified: file berubah dibanding commit terakhir.	Periksa dengan <code>git diff</code> .
Nama file merah + D (atau !)	Deleted (dihapus) / konflik.	Cek <code>git status</code> .
Nama file abu-abu	Diabaikan oleh .gitignore.	Normal (misal .venv, node_modules).
Garis hijau di tepi kiri baris	Baris baru (belum di-commit).	-
Garis biru di tepi kiri baris	Baris yang diubah.	-
Segitiga merah di tepi kiri	Ada baris yang dihapus di situ.	-

2.3 Warna di terminal

Terminal tidak punya aturan resmi, tetapi hampir semua program mengikuti kebiasaan yang sama:

Warna	Biasanya berarti	Contoh
Merah	Error / gagal / bahaya / penghapusan	Pesan error PowerShell, <code>ERROR</code> , <code>FAILED</code> , <code>git status</code> file yang belum di-add
Kuning	Peringatan (warning), perlu perhatian	<code>WARNING:</code> dari pip, <code>WARN</code> di npm, <code>[WARNING]</code>
Hijau	Sukses / OK / sudah siap	<code>[OK]</code> , <code>Successfully installed</code> , <code>git status</code> file yang sudah di-add, <code>INFO</code> hijau
Cyan / Biru	Informasi, judul, nama folder	Folder di <code>ls</code> Linux (biru), judul bagian script
Magenta / Ungu	Debug, nilai khusus, tautan	Log level <code>DEBUG</code> , nama cabang
Putih / abu	Keluaran biasa	Hasil <code>Get-Content</code>

Di script PowerShell lu sendiri warna dibuat dengan `Write-Host "teks" -ForegroundColor Red` (pilihan: Red, Green, Yellow, Cyan, Magenta, White, Gray, DarkRed, DarkGreen, DarkYellow, DarkCyan, dst). Itu sebabnya contoh `setup.ps1` memakai hijau untuk OK dan merah untuk GAGAL.

Warna di git (dalam terminal)

```
$ git status
Changes to be committed:      <- HIJAU : sudah di-add (siap di-commit)
    modified:   main.py
Changes not staged for commit: <- MERAH : berubah tapi belum di-add
    modified:   utils.py
Untracked files:              <- MERAH : file baru, belum dilacak
    catatan.txt

$ git diff
-   return a - b              <- MERAH, diawali "-" : baris dihapus
+   return a + b              <- HIJAU, diawali "+" : baris ditambah
@@ -3,2 +3,2 @@               <- CYAN : posisi baris di file
```

Anatomi error merah PowerShell

```
Get-Content : Cannot find path 'C:\x.txt' because it does not exist.
^ 1. PENYEBAB: baca baris ini dulu
At line:1 char:1
^ 2. LETAK: baris 1, karakter 1
+ Get-Content C:\x.txt
+ ~~~~~
^ 3. perintah bermasalah; tanda ~~~ menunjuk bagian yang salah
+ CategoryInfo          : ObjectNotFound: (C:\x.txt:String) [Get-Content], ItemNotFound...
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.GetContentCommand
^ 4. kode teknis (dua baris +): salin ke Google
```

Hampir selalu cukup membaca **baris pertama** (penyebab) dan baris yang ditandai ~~~~. Dua baris + **CategoryInfo** dan **FullyQualifiedErrorId** berguna untuk pencarian di internet.

Teks aneh seperti [32m atau ^[[31m di layar

Kalau terminal atau file log menampilkan karakter aneh seperti [32m atau \x1b[0m, itu bukan error. Itu **kode warna ANSI** yang tidak diterjemahkan oleh tampilan. **ESC** (kode 27, ditulis ^[, \x1b, atau \u001b) diikuti [, angka, dan m:

Kode	Arti	Kode	Arti
[0m	reset (kembali normal)	[1m	tebal
[31m	teks merah	[41m	latar merah
[32m	teks hijau	[42m	latar hijau
[33m	teks kuning	[43m	latar kuning
[34m	teks biru	[36m	teks cyan
[35m	teks magenta	[90m	teks abu-abu

Contoh: **ESC[32mOK ESC[0m** ditampilkan sebagai teks **OK** berwarna hijau. Untuk menghilangkan: gunakan terminal yang mendukung ANSI (Windows Terminal, VS Code) atau opsi **--no-color**.

2.4 Warna tingkat log (level)

Level	Arti	Warna umum	Perlu bertindak?
TRACE / DEBUG	Catatan sangat rinci untuk pengembang	abu / magenta	Tidak. Normal.
INFO	Kejadian normal	hijau / putih	Tidak.
WARN / WARNING	Ada yang janggal tapi masih jalan	kuning	Baca; mungkin perlu.
ERROR	Satu operasi gagal	merah	Ya , selidiki.
FATAL / CRITICAL	Program tidak bisa lanjut / mati	merah tebal	Ya , segera.

3. Terminal: tempat mengetik perintah

Terminal (atau *shell*) adalah jendela teks tempat lu mengetik perintah ke komputer, bukan klik mouse. Ada beberapa jenis, dan **simbolnya sedikit beda** antar jenis.

Terminal	Ada di	Ciri-ciri	Cara membuka
PowerShell	Windows	Tulisan awal PS C:\Users\NAMA> . Perintah gaya Verb-Noun (Get-ChildItem). Paling kuat di Windows.	Tombol Windows, ketik powershell , Enter. Atau klik kanan Start > Terminal.
CMD (Command Prompt)	Windows	Tulisan awal C:\Users\NAMA> . Lebih tua & terbatas. Perintah: dir, cd, copy.	Tombol Windows, ketik cmd , Enter.
Git Bash	Windows (setelah pasang Git)	Meniru Linux di Windows. Perintah: ls, cd, cp. Pakai / bukan \ .	Klik kanan di folder > Git Bash Here.
Terminal Linux/macOS (bash/zsh)	Linux, Mac	Awal baris user@komputer:~\$. Pemisah folder / .	Ctrl+Alt+T (Linux); Cmd+Spasi lalu ketik Terminal (Mac).
Terminal VS Code	Semua	Terminal di dalam editor, otomatis di folder proyek. Bisa pilih PowerShell/CMD/Bash.	Di VS Code tekan Ctrl+` (tanda backtick) atau menu Terminal > New Terminal.

3.1 Cara membaca prompt (tulisan di awal baris)

```
PS C:\Users\MyBook Hype\Documents> _
| |
| |
| |
| |
| +----- folder tempat lu berada sekarang
+----- "PS" = PowerShell
```

--- kursor: di sinilah lu mengetik
+---- tanda ">" artinya terminal siap menerima perintah

Setiap perintah dijalankan **di folder tempat lu berada**. Itu kenapa sering perlu **cd** (pindah folder) dulu sebelum menjalankan sesuatu. Kalau terminal bilang file tidak ditemukan, 90% penyebabnya: lu berada di folder yang salah.

3.2 Perintah dasar: PowerShell vs CMD vs Linux/Git Bash

Tujuan	PowerShell	CMD	Linux / Mac / Git Bash
Lihat isi folder	<code>Get-ChildItem</code> atau <code>ls</code> atau <code>dir</code>	<code>dir</code>	<code>ls</code> (<code>ls -la</code> lebih lengkap)
Pindah folder	<code>Set-Location D:\Proyek</code> atau <code>cd D:\Proyek</code>	<code>cd /d D:\Proyek</code>	<code>cd ~/proyek</code>
Naik satu folder	<code>cd ..</code>	<code>cd ..</code>	<code>cd ..</code>
Lihat lokasi sekarang	<code>Get-Location</code> atau <code>pwd</code>	<code>cd</code> (tanpa apa-apa)	<code>pwd</code>
Buat folder	<code>New-Item -ItemType Directory baru</code> atau <code>mkdir baru</code>	<code>mkdir baru</code>	<code>mkdir baru</code>
Buat file kosong	<code>New-Item catatan.txt</code>	<code>type nul > catatan.txt</code>	<code>touch catatan.txt</code>
Lihat isi file	<code>Get-Content a.txt</code> atau <code>cat a.txt</code>	<code>type a.txt</code>	<code>cat a.txt</code>
Salin file	<code>Copy-Item a.txt b.txt</code>	<code>copy a.txt b.txt</code>	<code>cp a.txt b.txt</code>
Pindah / ganti nama	<code>Move-Item a.txt b.txt</code>	<code>move a.txt b.txt</code>	<code>mv a.txt b.txt</code>
Hapus file	<code>Remove-Item a.txt</code>	<code>del a.txt</code>	<code>rm a.txt</code>
Hapus folder + isinya	<code>Remove-Item baru -Recurse -Force</code>	<code>rmdir /s /q baru</code>	<code>rm -rf baru</code>
Bersihkan layar	<code>Clear-Host</code> atau <code>cls</code>	<code>cls</code>	<code>clear</code>
Cetak teks	<code>Write-Host "halo"</code>	<code>echo halo</code>	<code>echo "halo"</code>
Cari teks dalam file	<code>Select-String "kata" a.txt</code>	<code>findstr kata a.txt</code>	<code>grep kata a.txt</code>
Cek program ada/tidak	<code>Get-Command python</code>	<code>where python</code>	<code>which python3</code>
Lihat variabel PATH	<code>\$env:Path</code>	<code>echo %PATH%</code>	<code>echo \$PATH</code>
Bantuan perintah	<code>Get-Help Copy-Item</code>	<code>copy /?</code>	<code>man cp</code> atau <code>cp --help</code>

3.3 Tombol keyboard yang wajib hafal di terminal

Tombol	Fungsi
Ctrl + C	Hentikan program yang sedang jalan / batalkan. Dipakai juga untuk menghentikan server. (Bukan 'copy' di terminal lama.)
Tab	Auto-lengkapi nama file/folder. Ketik <code>cd Doc</code> lalu Tab jadi <code>cd Documents</code> . Hemat waktu & mencegah salah ketik.
Panah atas / bawah	Ulangi perintah sebelumnya.
Ctrl + L (atau <code>cls</code>)	Bersihkan layar.
Klik kanan (di PowerShell lama)	Paste teks. Di terminal baru bisa Ctrl + V atau Ctrl + Shift + V.
Ctrl + R	Cari perintah lama (Linux/Mac/PowerShell terbaru).
Esc	Hapus seluruh baris yang sedang diketik (PowerShell).

3.4 Menjalankan file script

Jenis file	Cara menjalankan	Penjelasan
<code>.ps1</code> (PowerShell)	<code>.\setup.ps1</code>	Titik-garis miring = 'file di folder ini'. Wajib ada <code>.\</code> .
<code>.py</code> (Python)	<code>python main.py</code>	Di Mac/Linux sering <code>python3 main.py</code> .
<code>.js</code> (JavaScript)	<code>node contoh.js</code>	Perlu Node.js terpasang.
<code>.sh</code> (Bash)	<code>bash setup.sh</code> atau <code>./setup.sh</code>	Hanya Linux/Mac/Git Bash.
<code>.bat</code> / <code>.cmd</code>	<code>setup.bat</code>	Script CMD lama.

Masalah paling sering di Windows

Kalau `.\setup.ps1` menolak jalan dengan pesan *'running scripts is disabled on this system'*, itu aturan keamanan PowerShell (**ExecutionPolicy**).

Solusi aman, hanya untuk jendela terminal yang sedang terbuka (hilang saat ditutup):

`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

Setelah itu jalankan lagi `.\setup.ps1`. Hanya lakukan untuk script yang lu percaya.

3.5 Alur kerja khas dari nol (contoh)

```
# 1. Masuk ke folder proyek
Set-Location "C:\Users\MyBook Hype\Documents\app-clastrack-main"

# 2. Lihat isinya untuk memastikan lu di tempat yang benar
Get-ChildItem

# 3. Jalankan script persiapan
.\setup.ps1

# 4. Masuk ke backend, aktifkan environment Python, jalankan server
cd backend
.\.venv\Scripts\Activate.ps1
uvicorn main:app --port 8081

# 5. Buka browser ke http://localhost:8081
# Hentikan server dengan Ctrl + C
```

Perhatikan: tanda `#` di awal baris adalah komentar. Boleh ikut disalin, tidak dijalankan.

Trik hemat waktu

Ada spasi di nama folder (contoh: `MyBook Hype`)? **Apit dengan tanda kutip**: `cd "C:\Users\MyBook Hype"`. Tanpa kutip, terminal mengira itu dua kata berbeda.

Cara cepat: seret (drag) folder dari File Explorer ke jendela terminal, alamat lengkapnya otomatis terketik.

Di File Explorer, ketik `powershell` di kolom alamat folder lalu Enter: terminal langsung terbuka di folder itu.

3.6 Cara membuka terminal (semua cara)

Sistem	Cara	Hasil
Windows	Tekan tombol Windows , ketik powershell / cmd / terminal , Enter.	Terminal terbuka di folder pengguna (C:\Users\NAMA).
Windows	Tekan Win + R , ketik powershell (atau cmd , atau wt), Enter.	Sama. wt membuka Windows Terminal.
Windows	Tekan Win + X lalu pilih Terminal / Windows PowerShell .	Menu cepat. Ada pilihan (Admin) untuk hak administrator.
Windows	Di File Explorer, klik kolom alamat, ketik powershell (atau cmd), Enter.	Terminal terbuka langsung di folder itu . Cara paling praktis.
Windows 11	Klik kanan area kosong dalam folder > Open in Terminal .	Terminal di folder itu.
Windows 10	Shift + klik kanan area kosong dalam folder > Open PowerShell window here .	PowerShell di folder itu.
Git Bash	Klik kanan dalam folder > Git Bash Here (setelah Git dipasang).	Bash ala Linux di folder itu.
VS Code	Tekan Ctrl + ` atau menu Terminal > New Terminal.	Terminal di dalam VS Code, di folder proyek.
macOS	Cmd + Spasi , ketik Terminal , Enter.	zsh di folder home.
Linux	Ctrl + Alt + T (banyak distribusi).	bash di folder home.

Menjalankan sebagai Administrator (Windows): ketik **powershell** di menu Start, lalu tekan **Ctrl + Shift + Enter** (atau klik kanan > Run as administrator). Tanda berhasil: judul jendela berawalan **Administrator**. Hanya perlu untuk memasang program tingkat sistem, mengubah PATH sistem, atau mengakses file sistem. Di Linux/Mac, awali perintah dengan **sudo**.

Hati-hati dengan Admin

Jangan jalankan script yang tidak lu pahami sebagai Administrator. Admin boleh mengubah/menghapus apa saja di komputer.

3.7 Terminal hitam atau biru? Cara membedakan

Warna latar BUKAN penentu. Windows PowerShell bawaan (versi lama) memang berlatar biru, tetapi PowerShell 7, Windows Terminal, dan CMD berlatar hitam; semua warna bisa diubah pengguna. Penentu yang pasti adalah **tulisan di awal baris (prompt)** dan **judul jendela**.

Yang terlihat di awal baris	Ini adalah	Cara keluar
PS C:\Users\NAMA>	PowerShell (huruf PS di depan)	exit
C:\Users\NAMA>	CMD / Command Prompt (tanpa PS)	exit
NAMA@PC MINGW64 ~/proyek \$	Git Bash	exit
user@host:~\$	Bash Linux (atau WSL / Ubuntu di Windows)	exit
user@mac ~ %	zsh (macOS)	exit
(.venv) PS C:\...>	PowerShell + virtual environment Python aktif	deactivate (venv saja)
>>>	Bukan terminal biasa! Ini Python interaktif (REPL)	exit() atau Ctrl+Z lalu Enter (Windows) / Ctrl+D
>> (dua tanda) di PowerShell	Perintah belum selesai (ada kutip/kurung terbuka), PowerShell menunggu lanjutannya	Ctrl + C
> (satu tanda) di Node	Node.js REPL (JavaScript interaktif)	Ctrl+C dua kali / .exit

Yang terlihat di awal baris	Ini adalah	Cara keluar
<code>mongosh></code> atau <code>test></code>	Shell MongoDB	<code>exit</code>
<code>postgres=#</code>	psql (shell PostgreSQL)	<code>\q</code>
<code>mysql></code>	Shell MySQL	<code>exit</code>

Tes 5 detik bila masih ragu, ketik ini:

Ketik	Kalau di PowerShell	Kalau di CMD	Kalau di Bash
<code>\$PSVersionTable.PSVersion</code>	Menampilkan versi (5.1 = Windows PowerShell lama, 7.x = PowerShell baru)	Error / aneh	Error / aneh
<code>ver</code>	Error (tidak dikenal)	Menampilkan versi Windows	Error
<code>echo \$SHELL</code>	Baris kosong (variabel itu tidak ada)	Mencetak teks <code>\$SHELL</code> apa adanya	Menampilkan <code>/bin/bash</code> atau <code>/bin/zsh</code>

Windows PowerShell 5.1 vs PowerShell 7 vs Windows Terminal

Nama	File program	Apa itu
Windows PowerShell 5.1	<code>powershell.exe</code>	Bawaan Windows. Lama tapi selalu ada. Biasanya latar biru. Tidak punya <code>&&</code> dan <code> </code> .
PowerShell 7+	<code>pwsh.exe</code>	Versi baru, lintas platform (Windows/Linux/Mac). Punya <code>&&</code> , <code> </code> , <code>?:</code> . Dipasang terpisah: <code>winget install Microsoft.PowerShell</code> .
Windows Terminal	<code>wt.exe</code>	Aplikasi jendela (bukan shell) yang menampung banyak tab: PowerShell, CMD, Ubuntu, dsb. Warna & font bisa diatur. Bawaan Windows 11.
Command Prompt	<code>cmd.exe</code>	Shell klasik Windows. Perintah terbatas, tapi sangat kompatibel.
Git Bash	<code>bash.exe</code> (dari Git)	Bash ala Linux di Windows, memakai <code>/</code> sebagai pemisah folder.
WSL (Ubuntu)	<code>wsl.exe</code>	Linux sungguhan di dalam Windows. Folder Windows terlihat di <code>/mnt/c/....</code>

3.8 Perbedaan antar terminal (yang sering bikin salah)

Hal	PowerShell	CMD	Bash / zsh (Linux, Mac, Git Bash)
Pemisah folder	\ (/ juga diterima)	\	/
Pilihan 'folder ini'	.\file.ps1	file.bat / .\file	./file.sh
Besar-kecil huruf	tidak dibedakan	tidak dibedakan	dibedakan (File.txt beda file.txt)
Pakai variabel	\$nama	%nama%	\$nama
Isi variabel	\$nama = "Budi"	set nama=Budi	nama=Budi (tanpa spasi)
Variabel lingkungan	\$env:Path	%PATH%	\$PATH
Ubah variabel lingkungan sementara	\$env:PORT = "8081"	set PORT=8081	export PORT=8081
Pemisah dalam PATH	;	;	:
Komentar	#	REM atau ::	#
Lanjut ke baris berikutnya	backtick di akhir baris	^ di akhir baris	\ di akhir baris
Escape karakter khusus	backtick	^	\
Beberapa perintah sekaligus	a; b (7+: a && b)	a & b, a && b	a; b, a && b
Hasil perintah	objek (punya properti)	teks	teks
Tanda kutip	" " ganti variabel, ' ' apa adanya	" "	" " ganti variabel, ' ' apa adanya
Hapus layar	cls / Clear-Host	cls	clear
Akhir baris di file	CRLF (Windows)	CRLF	LF
File pengaturan pribadi	\$PROFILE	(tidak ada)	~/.bashrc / ~/.zshrc
Hak administrator	Run as administrator	Run as administrator	sudo perintah
Pemasang paket	winget, choco, scoop	sama	apt, dnf, brew, pacman

Pola kesalahan klasik: menyalin perintah Linux (`ls -la`, `rm -rf`, `export`, `/`) ke PowerShell, atau perintah PowerShell (`Remove-Item`, `$env:`) ke CMD/Bash. Kalau muncul 'not recognized', cek dulu terminal apa yang lu pakai (bab 3.7).

3.9 Aliran data: output, error, dan redirect

Setiap program punya tiga 'saluran': **stdin** (masukan), **stdout** (keluaran normal), dan **stderr** (keluaran error). Tanda `>` dll mengatur ke mana saluran itu dialirkan.

Simbol	Fungsi	Contoh
<code>></code>	Tulis stdout ke file (menimpa)	<code>python main.py > hasil.txt</code>
<code>>></code>	Tambahkan stdout ke akhir file	<code>echo baris >> log.txt</code>
<code>2></code>	Tulis error (stderr) ke file	<code>python main.py 2> error.txt</code>
<code>2>&1</code>	Gabungkan error ke aliran normal	<code>python main.py > semua.txt 2>&1</code>
<code>*></code> (PowerShell)	Semua aliran sekaligus	<code>.\setup.ps1 *> log.txt</code>
<code><</code>	Ambil masukan dari file (Bash/CMD)	<code>python main.py < input.txt</code>

Simbol	Fungsi	Contoh
	Kirim stdout program kiri sebagai masukan program kanan	<code>ls sort, Get-Process Sort-Object CPU</code>
<code>Tee-Object / tee</code>	Tampilkan di layar dan simpan ke file	<code>python main.py Tee-Object -FilePath log.txt</code>
<code>\$null / nul / /dev/null</code>	'Tempat sampah': buang keluaran	<code>perintah > \$null (PS), > nul (CMD), > /dev/null (Bash)</code>

3.10 PATH: kenapa 'not recognized' sering muncul

Saat lu mengetik `python`, terminal mencari file bernama `python.exe` di semua folder yang tercantum di variabel **PATH**, berurutan dari kiri. Kalau tidak ketemu, muncullah *'python is not recognized'*.

```
# Lihat isi PATH sebagai daftar rapi
$env:Path -split ";"          # PowerShell
echo %PATH%                  # CMD (tampil satu baris panjang)
echo $PATH | tr ":" "\n"     # Bash

# Cek program ditemukan di mana
Get-Command python           # PowerShell
where python                 # CMD
which python3                # Bash
```

Gejala	Penyebab	Solusi
<code>python</code> tidak dikenal setelah dipasang	Terminal dibuka sebelum instalasi; PATH belum diperbarui	Tutup semua terminal, buka lagi.
Masih tidak dikenal	Saat memasang, kotak <i>Add python.exe to PATH</i> tidak dicentang	Pasang ulang dan centang; atau tambahkan manual lewat <i>Environment Variables</i> .
<code>python</code> membuka Microsoft Store	Alias bawaan Windows menutupi Python asli	Settings > Apps > Advanced app settings > App execution aliases > matikan <i>python.exe</i> .
Versi salah yang jalan	Ada dua instalasi; yang di atas PATH yang menang	<code>Get-Command python -All</code> lalu rapikan urutan/hapus yang tidak dipakai.

4. winget dan pemasang paket lain

winget (Windows Package Manager) memasang, memperbarui, dan menghapus aplikasi lewat terminal. Tidak perlu membuka situs, mengunduh installer, dan klik Next berkali-kali. Cukup satu perintah. Sudah tersedia di Windows 11 dan Windows 10 versi baru (bagian dari *App Installer*).

4.1 Cek winget dan cara memperbaikinya

```
winget --version      # harus muncul angka versi, mis. v1.9.xxxx
winget --info         # info detail & lokasi
```

Kalau muncul *'winget is not recognized'*: buka Microsoft Store, cari **App Installer** (penerbit Microsoft Corporation), lalu **Update/Install**. Tutup dan buka lagi terminal. Perintah memasang butuh internet; sebagian paket akan memunculkan jendela izin Administrator (UAC): pilih **Yes**.

4.2 Perintah winget yang perlu dikenal

Tujuan	Perintah	Penjelasan
Cari paket	<code>winget search python</code>	Daftar paket beserta ID (kolom Id). ID itulah yang dipakai untuk install.
Lihat detail paket	<code>winget show Python.Python.3.12</code>	Versi, penerbit, lisensi, alamat unduh.
Pasang	<code>winget install --id Git.Git -e</code>	<code>--id</code> = pakai ID; <code>-e</code> (<code>--exact</code>) = ID harus persis (aman).
Pasang tanpa tanya-tanya	<code>winget install --id Git.Git -e --silent --accept-package-agreements --accept-source-agreements</code>	<code>--silent</code> tanpa jendela instalasi; dua <code>--accept-*</code> menyetujui lisensi otomatis (cocok untuk script).
Pasang versi tertentu	<code>winget install --id Python.Python.3.12 -e --version 3.12.4</code>	Kunci versi.
Pasang ke folder lain	<code>winget install --id X -e --location "D:\Apps\X"</code>	Tidak semua paket mendukung.
Daftar terpasang	<code>winget list</code>	Semua aplikasi di komputer (termasuk yang tidak dipasang lewat winget).
Filter daftar	<code>winget list python</code>	Hanya yang namanya cocok.
Cek pembaruan	<code>winget upgrade</code>	Daftar aplikasi yang punya versi baru.
Perbarui satu	<code>winget upgrade --id Git.Git -e</code>	
Perbarui semua	<code>winget upgrade --all</code>	Hati-hati: bisa banyak perubahan sekaligus.
Hapus	<code>winget uninstall --id Git.Git -e</code>	
Simpan daftar aplikasi	<code>winget export -o apps.json</code>	Menyimpan daftar aplikasi terpasang ke file JSON.
Pasang semuanya dari daftar	<code>winget import -i apps.json</code>	Berguna saat menyiapkan komputer baru.
Sumber paket	<code>winget source list / winget source update</code>	Muat ulang katalog bila pencarian aneh.
Bantuan	<code>winget install --help</code>	Semua opsi sebuah perintah.

Singkatan opsi

Panjang	Pendek	Arti
<code>--id</code>	-	Cocokkan berdasarkan ID
<code>--exact</code>	<code>-e</code>	Cocok persis, bukan 'mengandung'
<code>--silent</code>	<code>-h</code>	Tanpa antarmuka
<code>--interactive</code>	<code>-i</code>	Tampilkan installer (bisa pilih opsi)
<code>--version</code>	<code>-v</code>	Versi tertentu
<code>--source</code>	<code>-s</code>	Pilih sumber (<code>winget</code> atau <code>msstore</code>)
<code>--scope</code>	-	<code>user</code> (hanya akun ini) atau <code>machine</code> (semua pengguna, butuh admin)

4.3 ID paket yang sering dipakai programmer

Format ID: `Penerbit.NamaPaket`. ID dapat berubah, jadi verifikasi dulu dengan `winget search nama` sebelum memakainya.

Aplikasi	ID	Keterangan
Git	<code>Git.Git</code>	Git untuk Windows (termasuk Git Bash)
Python 3.12	<code>Python.Python.3.12</code>	Ada juga <code>Python.Python.3.13</code> , dst.
Node.js (LTS)	<code>OpenJS.NodeJS.LTS</code>	Termasuk <code>npm</code>
VS Code	<code>Microsoft.VisualStudioCode</code>	Editor
PowerShell 7	<code>Microsoft.PowerShell</code>	Perintah: <code>pwsh</code>
Windows Terminal	<code>Microsoft.WindowsTerminal</code>	
GitHub CLI	<code>GitHub.cli</code>	Perintah <code>gh</code>
MongoDB Compass	<code>MongoDB.Compass.Full</code>	Antarmuka grafis MongoDB
MongoDB Server	<code>MongoDB.Server</code>	Server database MongoDB
Postman	<code>Postman.Postman</code>	Menguji API
7-Zip	<code>7zip.7zip</code>	
Notepad++	<code>Notepad++.Notepad++</code>	

4.4 Contoh script menyiapkan komputer programmer

```
# siapkan_pc.ps1 - memasang alat pengembangan lewat winget
$app = @(
    "Git.Git",
    "Python.Python.3.12",
    "OpenJS.NodeJS.LTS",
    "Microsoft.VisualStudioCode"
)

foreach ($a in $app) {
    Write-Host "==" Memasang $a == " -ForegroundColor Cyan
    winget install --id $a -e --silent --accept-package-agreements --accept-source-agreements
    if ($LASTEXITCODE -ne 0) {
        Write-Host "Gagal / sudah terpasang: $a (kode $LASTEXITCODE)" -ForegroundColor Yellow
    }
}

Write-Host "Selesai. TUTUP lalu BUKA ULANG terminal agar PATH diperbarui." -ForegroundColor Green
```

Cara kerja: `@( ... )` membuat daftar ID, `foreach` memasang satu per satu, `$LASTEXITCODE` berisi kode keluar `winget` terakhir (0 = sukses), `-ne` = tidak sama dengan. Jalankan dengan `.\siapkan_pc.ps1` (kalau diblokir, lihat 3.4).

4.5 Masalah umum winget

Pesan / gejala	Penyebab	Solusi
No package found matching input criteria	ID atau nama salah	<code>winget search kata</code> ; salin ID persis dari kolom Id.
Multiple packages found matching input criteria	Nama terlalu umum	Pakai <code>--id X -e</code> .
Diminta menyetujui perjanjian sumber (msstore)	Pertama kali memakai sumber Microsoft Store	Ketik <code>Y</code> , atau tambah <code>--accept-source-agreements</code> .
Installer hash does not match	Berkas unduhan berbeda dari yang tercatat (mungkin pembaruan paket)	<code>winget source update</code> , coba lagi nanti. Jangan memaksa melewati pemeriksaan.
Instalasi diam saja / lama	Menunggu izin UAC di balik jendela	Cek ikon perisai di taskbar.
Program terpasang tapi tidak dikenali terminal	PATH belum dimuat ulang	Tutup semua terminal, buka lagi.
0x8a15... (kode heksadesimal)	Kode error internal winget	Salin kodenya ke Google bersama kata <code>winget</code> .

4.6 Pemasang paket lain (perbandingan)

Pemasang	Untuk apa	Cari	Pasang	Hapus	Perbarui semua
winget	Aplikasi Windows	<code>winget search x</code>	<code>winget install --id X -e</code>	<code>winget uninstall --id X -e</code>	<code>winget upgrade --all</code>
choco	Aplikasi Windows (pihak ketiga)	<code>choco search x</code>	<code>choco install x</code>	<code>choco uninstall x</code>	<code>choco upgrade all</code>
scoop	Alat CLI Windows (tanpa admin)	<code>scoop search x</code>	<code>scoop install x</code>	<code>scoop uninstall x</code>	<code>scoop update *</code>
brew	macOS / Linux	<code>brew search x</code>	<code>brew install x</code>	<code>brew uninstall x</code>	<code>brew upgrade</code>
apt	Ubuntu / Debian	<code>apt search x</code>	<code>sudo apt install x</code>	<code>sudo apt remove x</code>	<code>sudo apt update && sudo apt upgrade</code>
dnf	Fedora / RHEL	<code>dnf search x</code>	<code>sudo dnf install x</code>	<code>sudo dnf remove x</code>	<code>sudo dnf upgrade</code>
pacman	Arch Linux	<code>pacman -Ss x</code>	<code>sudo pacman -S x</code>	<code>sudo pacman -R x</code>	<code>sudo pacman -Syu</code>
pip	Library Python	<code>pip search*</code> / PyPI	<code>pip install x</code>	<code>pip uninstall x</code>	<code>pip install -U x</code>
npm	Library JavaScript	situs npmjs.com	<code>npm install x</code>	<code>npm uninstall x</code>	<code>npm update</code>

* `pip search` sudah dinonaktifkan; cari nama paket di pypi.org.

5. Simbol-simbol PowerShell

Simbol	Nama	Fungsi	Contoh
\	backslash	Pemisah folder di Windows	C:\Users\NAMA\Documents
/	slash	Pemisah folder di web & Linux; juga pembagian	http://localhost:8081/api, 10 / 2
.	titik	Satu titik = folder ini; .. = folder di atas; juga untuk mengambil isi objek	.\setup.ps1, cd .., \$f.Name
" "	kutip dua	Teks; variabel di dalamnya diganti nilainya	"Halo \$nama" -> Halo Budi
' '	kutip satu	Teks apa adanya, variabel tidak diganti	'Halo \$nama' -> Halo \$nama
[]	kurung siku	Daftar (array), ambil item ke-sekian, atau tipe data	\$a[0], [int]"5"
{ }	kurung kurawal	Blok perintah: isi if, foreach, fungsi	if (\$x) { Write-Host "ya" }
()	kurung biasa	Mengelompokkan / kondisi; dikerjakan duluan	if (-not (Test-Path \$p))
\$	dolar	Awalan nama variabel	\$Root = "C:\\"
\$()	sub-ekspresi	Menjalankan ekspresi di dalam teks	"Total: \$(\$a.Count)"
@()	array	Membuat daftar	@("a","b","c")
@{ }	hashtable	Pasangan nama = nilai (seperti kamus)	@{ Nama="Budi"; Umur=16 }
=	sama dengan	Mengisi variabel (bukan membandingkan!)	\$x = 5
;	titik koma	Pemisah perintah dalam satu baris	cd ..; ls
:	titik dua	Penanda drive, atau pasangan di JSON; di PowerShell juga \$env:Path	C:, \$env:Path
	pipa (pipe)	Hasil perintah kiri dikirim ke kanan	ls Sort-Object Length
>	redirect	Simpan hasil ke file (menimpa)	ls > daftar.txt
>>	redirect tambah	Simpan hasil ke file (menambah di akhir)	echo baris >> log.txt
-	strip	Awalan opsi/parameter & operator	-Recurse -Force, -eq
#	pagar	Komentar satu baris	# ini catatan
<# #>	komentar blok	Komentar banyak baris	<# catatan panjang #>
`	backtick	Lanjut ke baris berikutnya, atau karakter khusus	`n = baris baru, `" = kutip literal
&	ampersand (call)	Menjalankan perintah/jalur yang disimpan dalam teks	& ".\env\Scripts\Activate.ps1"
*	bintang (wildcard)	Mewakili apa saja	*.txt = semua file berakhiran .txt
?	tanya (wildcard)	Mewakili satu karakter apa saja	file?.txt = file1.txt, fileA.txt
~	tilde	Folder 'home' pengguna	cd ~
%	persen	Sisa bagi, atau alias ForEach-Object	10 % 3 = 1
!	seru	Sama dengan -not (tidak)	if (!\$ada) { ... }

5.1 Operator perbandingan & logika PowerShell

PowerShell **tidak** memakai **==** atau **>** untuk membandingkan, melainkan kata berawalan strip:

Operator	Arti	Contoh -> hasil
<code>-eq</code>	sama dengan (equal)	<code>5 -eq 5 -> True</code>
<code>-ne</code>	tidak sama (not equal)	<code>5 -ne 3 -> True</code>
<code>-gt / -ge</code>	lebih besar / lebih besar atau sama	<code>7 -gt 5 -> True</code>
<code>-lt / -le</code>	lebih kecil / lebih kecil atau sama	<code>3 -lt 5 -> True</code>
<code>-like</code>	cocok pola wildcard	<code>"budi" -like "b*" -> True</code>
<code>-match</code>	cocok pola regex	<code>"abc123" -match "\d+" -> True</code>
<code>-contains / -in</code>	daftar memuat / ada di dalam daftar	<code>@(1,2,3) -contains 2 -> True</code>
<code>-and</code>	dan (keduanya harus benar)	<code>(\$a -gt 1) -and (\$b -lt 9)</code>
<code>-or</code>	atau (salah satu benar)	<code>\$a -or \$b</code>
<code>-not</code>	kebalikan	<code>-not \$true -> False</code>
<code>+ - * /</code>	tambah, kurang, kali, bagi	<code>2 + 3 * 4 -> 14</code>
<code>++ / --</code>	tambah 1 / kurang 1	<code>\$i++</code>
<code>+=</code>	tambah ke nilai sendiri	<code>\$x += 5</code> sama dengan <code>\$x = \$x + 5</code>

5.2 Variabel & nilai khusus PowerShell

Nama	Arti
<code>\$true / \$false</code>	Benar / salah.
<code>\$null</code>	Kosong / tidak ada nilai.
<code>\$_</code>	Item yang sedang diproses dalam loop/pipa (<code>ForEach-Object</code> , <code>Where-Object</code>).
<code>\$env:NAMA</code>	Variabel lingkungan Windows, misal <code>\$env:USERNAME</code> , <code>\$env:Path</code> , <code>\$env:USERPROFILE</code> .
<code>\$PSScriptRoot</code>	Folder tempat file script itu berada.
<code>\$?</code>	Apakah perintah terakhir berhasil (True/False).
<code>\$LASTEXITCODE</code>	Kode keluar program terakhir (0 = sukses). Lihat bab angka.
<code>\$args</code>	Argumen yang dikirim ke script/fungsi.
<code>\$PROFILE</code>	Lokasi file pengaturan PowerShell pribadi.

5.3 Pola perintah: Verb-Noun

Hampir semua perintah PowerShell (disebut **cmdlet**) berbentuk **Kata-Kerja + Kata-Benda**, jadi mudah ditebak:

Verb	Arti	Contoh cmdlet
Get	ambil / lihat	<code>Get-ChildItem</code> , <code>Get-Content</code> , <code>Get-Process</code> , <code>Get-Command</code>
Set	ubah / tetapkan	<code>Set-Location</code> , <code>Set-Content</code> , <code>Set-ExecutionPolicy</code>
New	buat baru	<code>New-Item</code> , <code>New-Object</code>
Remove	hapus	<code>Remove-Item</code> , <code>Remove-Variable</code>
Copy / Move	salin / pindah	<code>Copy-Item</code> , <code>Move-Item</code>
Test	periksa (hasil True/False)	<code>Test-Path</code> , <code>Test-Connection</code>
Write	tampilkan keluaran	<code>Write-Host</code> , <code>Write-Output</code> , <code>Write-Error</code>

Verb	Arti	Contoh cmdlet
Invoke	jalankan	Invoke-WebRequest, Invoke-Expression
Select / Where / Sort	pilih / saring / urutkan	Select-Object, Where-Object, Sort-Object

Parameter selalu diawali strip: **Remove-Item .\x -Recurse -Force**. **Tips:** ketik **Get-Help NamaPerintah -Examples** untuk melihat contoh resmi.

5.4 Contoh script PowerShell yang sudah jadi

Contoh A: setup.ps1 - memeriksa program, membuat virtual environment Python, memasang library.

```
# setup.ps1 - menyiapkan proyek (contoh gaya KelasTrack)
$ErrorActionPreference = "Stop" # berhenti kalau ada error
$Root = Split-Path -Parent $MyInvocation.MyCommand.Path

function Has($nama) {
    return [bool](Get-Command $nama -ErrorAction SilentlyContinue)
}

Write-Host "== Cek program ==" -ForegroundColor Cyan
foreach ($p in @("python", "node", "git")) {
    if (Has $p) {
        Write-Host "[OK] $p ditemukan" -ForegroundColor Green
    } else {
        Write-Host "[GAGAL] $p belum terpasang" -ForegroundColor Red
        exit 1
    }
}

$Backend = Join-Path $Root "backend"
Set-Location $Backend

if (-not (Test-Path ".venv")) {
    Write-Host "Membuat virtual environment..."
    python -m venv .venv
}

& ".\venv\Scripts\Activate.ps1" # aktifkan venv
python -m pip install --upgrade pip
pip install -r requirements.txt

Write-Host "Selesai! Jalankan: uvicorn main:app --port 8081" -ForegroundColor Green
```

Cara kerja singkat: **function Has** membuat 'alat cek' sendiri. **foreach** mengulang daftar program. **if/else** memberi hasil OK atau GAGAL. **Join-Path** menyambung alamat folder dengan aman. **-not (Test-Path ".venv")** artinya 'kalau folder .venv BELUM ada'.

Contoh B: rapikan_file.ps1 - mencari, mengurutkan, dan menyalin file.

```
# rapikan_file.ps1 - contoh kerja dengan file & folder
$folder = "C:\Users\NAMA\Documents\Tugas"

# 1. ambil semua file .txt, urutkan dari yang terbesar
$daftar = Get-ChildItem -Path $folder -Filter "*.txt" -Recurse |
    Sort-Object Length -Descending

# 2. tampilkan nama & ukuran (KB)
foreach ($f in $daftar) {
    $kb = [math]::Round($f.Length / 1KB, 1)
    Write-Host "$($f.Name) - $kb KB"
}

# 3. hitung ada berapa file
Write-Host "Total: $($daftar.Count) file"

# 4. buat folder backup kalau belum ada, lalu salin
$backup = Join-Path $folder "backup"
```

```
if (-not (Test-Path $backup)) { New-Item -ItemType Directory -Path $backup | Out-Null }
$daftar | Copy-Item -Destination $backup
```

Contoh C: array, hashtable, perulangan

```
# array, hashtable, perulangan
$buah = @("apel", "jeruk", "mangga")           # array
Write-Host $buah[0]                             # apel (indeks mulai 0)
Write-Host $buah.Count                          # 3

$umur = @{ Budi = 16; Sari = 15 }               # hashtable (nama = nilai)
Write-Host $umur["Budi"]                        # 16

foreach ($b in $buah) { Write-Host "Saya suka $b" }

for ($i = 1; $i -le 3; $i++) { Write-Host "Hitungan ke-$i" }

1..5 | ForEach-Object { $_ * 2 }                 # 2 4 6 8 10 ($_ = item saat ini)
```

6. Python

Python adalah bahasa pemrograman yang mudah dibaca. File berakhiran `.py`. Cek terpasang atau belum dengan: `python --version`.

6.1 Simbol-simbol Python

Simbol	Nama	Fungsi	Contoh
#	pagar	Komentar (tidak dijalankan)	# hitung total
""" """	kutip tiga	Teks banyak baris / dokumentasi fungsi	"""Penjelasan fungsi"""
" " atau ' '	kutip	Teks (string). Boleh dua atau satu, hasil sama	"halo", 'halo'
=	sama dengan	Mengisi variabel	umur = 16
==	sama dengan (banding)	Membandingkan dua nilai	umur == 16 -> True
[]	kurung siku	List (daftar), atau ambil item ke-n	a = [10, 20, 30], a[0] -> 10
{ }	kurung kurawal	Dictionary (kamus) / set; juga isi f-string	{"nama": "Budi"}, f"{x}"
()	kurung biasa	Memanggil fungsi, tuple, atau mengelompokkan	print(5), (1, 2)
:	titik dua	Penutup baris if/for/def/class (wajib!), juga pasangan kunci-nilai & potongan list	if x > 3:, a[1:3]
,	koma	Pemisah item	[1, 2, 3]
.	titik	Mengambil bagian dari sebuah objek	teks.upper(), os.path
\	backslash	Karakter khusus dalam teks; lanjut baris	"baris1\nbaris2"
*	bintang	Kali; ** pangkat; *args banyak argumen	3 * 4, 2 ** 3 -> 8
//	bagi bulat	Hasil bagi tanpa koma	7 // 2 -> 3
%	persen (modulo)	Sisa bagi	7 % 2 -> 1
@	at	Dekorator: menambah fungsi tambahan pada fungsi di bawahnya	@app.get("/")
_	garis bawah	Awalan nama 'privat', atau variabel yang diabaikan	_rahasia, for _ in range(3)
__init__	dunder	Nama khusus Python (dua garis bawah di kiri-kanan)	__init__, __name__, __str__
->	panah	Petunjuk tipe nilai keluaran fungsi (opsional)	def f() -> int:
:=	walrus	Mengisi variabel sekaligus memakainya	if (n := len(a)) > 3:

Aturan terpenting Python

Python memakai **indentasi** (menjorok ke dalam, 4 spasi) untuk menandai isi blok. Bukan kurung kurawal. Satu spasi saja salah → **IndentationError**. Jangan campur Tab dan spasi. Gunakan 4 spasi konsisten. Setiap baris **if**, **for**, **while**, **def**, **class**, **try**, **else** **harus diakhiri titik dua **:**.

```
umur = 16
if umur >= 17:           # <- diakhiri titik dua
    print("Dewasa")      # <- menjorok 4 spasi = isi dari if
else:
    print("Belum")       # <- juga menjorok
print("selesai")         # <- tidak menjorok = di luar if, selalu jalan
```

6.2 Tipe data

Tipe	Arti	Contoh	Catatan
<code>int</code>	bilangan bulat	7, -3, 1000	<code>int("42")</code> mengubah teks jadi angka
<code>float</code>	bilangan desimal (pakai titik)	3.14, 0.5	Titik, bukan koma. <code>1e3</code> = 1000.0
<code>str</code>	teks	"halo"	<code>str(5)</code> mengubah angka jadi teks
<code>bool</code>	benar/salah	True, False	Huruf depan besar
<code>None</code>	kosong	None	Padanan <code>\$null</code> PowerShell
<code>list</code>	daftar berurutan, bisa diubah	[1, 2, 3]	<code>a.append(4)</code> menambah item
<code>tuple</code>	daftar yang tidak bisa diubah	(1, 2)	
<code>dict</code>	kamus: kunci -> nilai	{"a": 1, "b": 2}	<code>d["a"]</code> -> 1
<code>set</code>	kumpulan unik, tanpa urutan	{1, 2, 3}	Duplikat otomatis hilang

6.3 Operator Python

Jenis	Operator	Contoh -> hasil
Hitung	+ - * / // % **	10 / 4 -> 2.5, 10 // 4 -> 2, 10 % 4 -> 2, 2 ** 5 -> 32
Bandingkan	== != > < >= <=	5 == 5 True, 5 != 3 True
Logika	and or not	x > 1 and x < 9
Keanggotaan	in not in	"a" in "kata" -> True
Identitas	is is not	x is None
Tambah ke diri sendiri	+= -= *= /=	total += 5
Format teks (f-string)	f"... {variabel} ..."	f"Umur {umur}" -> Umur 16
Format angka dalam f-string	{x:.2f} {x:>8} {x:05d}	{3.14159:.2f} -> 3.14; {7:05d} -> 00007

6.4 Fungsi bawaan yang sering dipakai

Fungsi	Kegunaan	Contoh
<code>print()</code>	tampilkan ke layar	<code>print("halo")</code>
<code>input()</code>	minta ketikan pengguna	<code>nama = input("Nama: ")</code>
<code>len()</code>	panjang/jumlah item	<code>len([1, 2, 3])</code> -> 3
<code>range()</code>	deret angka	<code>range(3)</code> -> 0,1,2; <code>range(1, 4)</code> -> 1,2,3
<code>sum()</code> <code>min()</code> <code>max()</code>	jumlah, terkecil, terbesar	<code>sum([1, 2, 3])</code> -> 6
<code>sorted()</code>	urutkan	<code>sorted([3, 1, 2])</code> -> [1,2,3]
<code>int()</code> <code>float()</code> <code>str()</code>	ubah tipe	<code>float("2.5")</code> -> 2.5
<code>round()</code>	pembulatan	<code>round(3.14159, 2)</code> -> 3.14
<code>open()</code>	buka file	<code>open("a.txt", "r")</code>
<code>type()</code>	cek tipe	<code>type(5)</code> -> int

6.5 Virtual environment & pip (sering muncul di proyek)

venv (virtual environment, biasanya folder **.venv**) adalah 'kotak terpisah' berisi library Python milik satu proyek, supaya tidak bentrok dengan proyek lain. **pip** adalah pemasang library.

```
python -m venv .venv                # buat venv (sekali saja)

.\.venv\Scripts\Activate.ps1        # AKTIFKAN di PowerShell (Windows)
.venv\Scripts\activate.bat          # aktifkan di CMD
source .venv/bin/activate            # aktifkan di Linux / Mac / Git Bash

pip install fastapi                  # pasang satu library
pip install -r requirements.txt      # pasang semua library dari daftar
pip list                             # lihat yang terpasang
pip freeze > requirements.txt        # simpan daftar library ke file
deactivate                           # keluar dari venv
```

Setelah aktif, awal prompt berubah menjadi **(.venv) PS C:\...>**. Tanda **(.venv)** itu pertanda venv sedang aktif.

6.6 Contoh program Python yang sudah jadi

Contoh 1: hitung nilai siswa (list, dictionary, fungsi, if/elif/else, loop, f-string)

```
# contoh_nilai.py - menghitung rata-rata nilai siswa
nilai_siswa = {
    "Budi": [80, 75, 90],
    "Sari": [95, 88, 92],
    "Andi": [60, 70, 65],
}

def rata_rata(daftar):
    """Mengembalikan rata-rata dari sebuah list angka."""
    return sum(daftar) / len(daftar)

def predikat(nilai):
    if nilai >= 90:
        return "A"
    elif nilai >= 75:
        return "B"
    elif nilai >= 60:
        return "C"
    else:
        return "D"

for nama, daftar in nilai_siswa.items():
    rata = rata_rata(daftar)
    print(f"{nama:<6} rata-rata = {rata:6.2f} predikat = {predikat(rata)}")
```

Hasil yang muncul:

```
Budi rata-rata = 81.67 predikat = B
Sari rata-rata = 91.67 predikat = A
Andi rata-rata = 65.00 predikat = C
```

Arti **{nama:<6}** = rata kiri lebar 6 karakter; **{rata:6.2f}** = angka desimal, lebar 6, 2 angka di belakang koma.

Contoh 2: simpan & baca file JSON

```
# contoh_json.py - menyimpan & membaca data dari file JSON
import json
from pathlib import Path

file_data = Path("siswa.json")

siswa = [
    {"nama": "Budi", "kelas": "10A", "hadir": True},
    {"nama": "Sari", "kelas": "10B", "hadir": False},
```

```

]

# tulis ke file
with open(file_data, "w", encoding="utf-8") as f:
    json.dump(siswa, f, indent=2, ensure_ascii=False)

# baca lagi
with open(file_data, "r", encoding="utf-8") as f:
    hasil = json.load(f)

for s in hasil:
    status = "hadir" if s["hadir"] else "tidak hadir"
    print(s["nama"], "-", s["kelas"], "-", status)

```

Contoh 3: class (membuat objek)

```

# contoh_class.py - class = cetakan untuk membuat objek
class Siswa:
    def __init__(self, nama, kelas):
        self.nama = nama          # self = objek itu sendiri
        self.kelas = kelas
        self.absen = []          # daftar tanggal tidak hadir

    def catat_absen(self, tanggal):
        self.absen.append(tanggal)

    def jumlah_absen(self):
        return len(self.absen)

    def __str__(self):
        # dipanggil oleh print()
        return f"{self.nama} ({self.kelas}) - absen {self.jumlah_absen()}x"

budi = Siswa("Budi", "10A")
budi.catat_absen("2026-09-01")
budi.catat_absen("2026-09-15")
print(budi)

```

Contoh 4: menangani error dengan try/except

```

# contoh_error.py - try / except supaya program tidak langsung mati
def bagi(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("Tidak bisa dibagi nol!")
        return None
    finally:
        print("selesai menghitung")

print(bagi(10, 2))    # 5.0
print(bagi(10, 0))    # None
angka = int("42")    # teks -> bilangan bulat
print(angka + 8)      # 50

```

Contoh 5: API web mini dengan FastAPI (kerangka seperti backend yang punya alamat /api/settings)

```

# main.py - API sederhana dengan FastAPI (contoh; perlu: pip install fastapi uvicorn)
from fastapi import FastAPI

app = FastAPI()

settings = {"nama_sekolah": "SMA Contoh", "tahun": 2026}

@app.get("/")
def beranda():
    return {"pesan": "Server hidup"}

@app.get("/api/settings")
def baca_settings():

```

```
    return settings

@app.get("/api/siswa/{siswa_id}")
def baca_siswa(siswa_id: int):
    return {"id": siswa_id, "nama": f"Siswa ke-{siswa_id}"}

# Jalankan dengan: uvicorn main:app --reload --port 8081
# Buka di browser: http://localhost:8081/api/settings
```

Di contoh 5: `@app.get("/api/settings")` artinya 'kalau ada yang membuka alamat `/api/settings`, jalankan fungsi di bawahnya'. `{siswa_id}` di alamat adalah bagian yang bisa berubah. `--reload` memuat ulang server otomatis saat kode diubah. `8081` adalah nomor port.

7. Arti angka-angka dalam coding

Angka di dunia coding tidak selalu berarti 'jumlah'. Ini arti yang paling sering ditemui:

7.1 Hitungan mulai dari 0 (indeks)

Hampir semua bahasa (Python, JavaScript, PowerShell, C, Java) menghitung posisi item **mulai dari 0**, bukan 1.

```
buah = ["apel", "jeruk", "mangga"]
#      0      1      2      <- nomor posisi (indeks)
#      -3     -2     -1     <- dari belakang (Python)

buah[0]    # "apel"      item pertama
buah[2]    # "mangga"    item ketiga
buah[-1]   # "mangga"    item TERAKHIR
buah[0:2]  # ["apel", "jeruk"] potong: dari 0 sampai SEBELUM 2
len(buah)  # 3           jumlah item, tapi indeks terakhir = 2
range(3)   # 0, 1, 2     3 angka, mulai 0, berhenti sebelum 3
```

7.2 Sistem bilangan: desimal, biner, heksadesimal

Sistem	Digit	Contoh	Nilai desimal	Dipakai untuk
Desimal (basis 10)	0-9	255	255	Angka sehari-hari
Biner (basis 2)	0 dan 1	0b11111111 (Python)	255	Cara kerja komputer, bit
Oktal (basis 8)	0-7	0o377 (Python)	255	Izin file Linux (chmod 755)
Heksadesimal (basis 16)	0-9, A-F	0xFF / #FF0000	255	Warna, alamat memori, MAC address

Contoh warna #2F6DB5 : 2F = merah (47), 6D = hijau (109), B5 = biru (181). Python: `int("FF", 16)` -> 255, `bin(5)` -> '0b101', `hex(255)` -> '0xff'.

Satuan data	Besar
1 bit	satu angka 0 atau 1
1 byte	8 bit (nilai 0-255)
1 KB / MB / GB	sekitar 1.000 / 1.000.000 / 1.000.000.000 byte (komputer menghitung 1024)
PowerShell 1KB, 1MB, 1GB	Konstanta ukuran bawaan, misal <code>\$f.Length / 1MB</code>

7.3 Exit code (kode keluar) - 0 berarti sukses

Setiap program yang selesai mengembalikan satu angka ke terminal:

Angka	Arti
0	Sukses, tidak ada masalah.
1 (atau angka lain bukan 0)	Ada masalah / gagal. Angkanya bisa dipilih pembuat program.
<code>exit 1</code> di script	Hentikan script dan laporkan gagal.
<code>exit 0</code>	Hentikan script dan laporkan sukses.

Cek exit code terakhir: PowerShell `$LASTEXITCODE`, CMD `echo %ERRORLEVEL%`, Bash `echo $?`.

7.4 Port - angka setelah titik dua di alamat

Komputer bisa menjalankan banyak program jaringan sekaligus. **Port** adalah nomor 'pintu' tiap program. Alamat `http://localhost:8081/api/settings` dibaca:

```

http://localhost:8081/api/settings
|           |           |           |
|           |           | +-- jalur (path): bagian mana yang diminta
|           | +----- port 8081: pintu nomor 8081
| +----- host: "localhost" = komputer lu sendiri (sama dengan 127.0.0.1)
+----- protokol: http (https = versi aman)

```

Port	Biasanya untuk
80	Web biasa (http), tidak perlu ditulis di alamat
443	Web aman (https), tidak perlu ditulis
22	SSH (akses server jarak jauh)
21	FTP
3000	Server pengembangan React / Node
5173	Vite (alat pengembangan web)
5432	PostgreSQL (database)
3306	MySQL / MariaDB (database)
27017	MongoDB (database)
8000, 8080, 8081	Server pengembangan umum (Python/FastAPI/Django/Java)

Port 0-1023 khusus sistem; untuk proyek sendiri pakai 3000-9999. Error **'port already in use'** berarti port itu sedang dipakai program lain: tutup program itu (Ctrl+C di terminalnya) atau ganti nomor port.

7.5 Alamat IP

Alamat	Arti
127.0.0.1 / localhost	Komputer lu sendiri (loopback).
0.0.0.0	'Semua alamat' - server bisa diakses dari perangkat lain di jaringan.
192.168.x.x, 10.x.x.x	Alamat jaringan lokal (rumah/kantor/sekolah), bukan internet.

7.6 Versi: 3.11.4

Format versi **MAYOR.MINOR.PATCH**. Contoh Python **3.11.4**: 3 = mayor (perubahan besar, bisa tidak kompatibel), 11 = minor (fitur baru), 4 = patch (perbaikan bug). Di **requirements.txt/package.json** ada tanda khusus:

Penulisan	Arti
fastapi==0.110.0	Harus tepat versi itu (Python/pip).
fastapi>=0.100	Versi 0.100 atau lebih baru.
"react": "^18.2.0"	Boleh naik minor/patch (18.x.x), jangan lompat ke 19 (npm).
"react": "~18.2.0"	Boleh naik patch saja (18.2.x) (npm).

7.7 Kode status HTTP (angka balasan server)

Kode	Nama	Maksud
200	OK	Berhasil.
201	Created	Berhasil membuat data baru.
204	No Content	Berhasil, tidak ada isi balasan.
301 / 302	Redirect	Dialihkan ke alamat lain.
400	Bad Request	Permintaan salah format / data tidak lengkap.
401	Unauthorized	Belum login / kredensial salah.
403	Forbidden	Sudah login, tapi tidak punya izin.
404	Not Found	Alamat / data tidak ada. Cek ejaan URL.
422	Unprocessable	Format benar tapi isi tidak valid (sering di FastAPI).
429	Too Many Requests	Terlalu sering meminta, pelan-pelan.
500	Server Error	Bug / kesalahan di sisi server. Lihat log server.
502 / 503	Bad Gateway / Unavailable	Server mati atau sedang sibuk.

Pola: **2xx** = sukses, **3xx** = dialihkan, **4xx** = salah di pihak peminta, **5xx** = salah di pihak server.

7.8 Angka lain yang sering ditemui

Angka / bentuk	Arti	Contoh
-1	'tidak ketemu' / 'terakhir' / 'tak terbatas'	<code>buah[-1]; "abc".find("z") -> -1</code>
1 dan 0	Sering dipakai sebagai True / False	<code>DEBUG=1</code>
1e3, 2.5e-4	Notasi ilmiah: 1×10^3 dan $2,5 \times 10^{-4}$	<code>1e3 -> 1000.0</code>
1_000_000	Garis bawah pemisah ribuan agar mudah dibaca	Python: sama dengan 1000000
755, 644	Izin file Linux (pemilik/grup/lainnya: baca=4, tulis=2, jalan=1)	<code>chmod 755 setup.sh</code>
1700000000	Epoch / Unix time: detik sejak 1 Jan 1970	<code>time.time()</code>
2026-10-01	Format tanggal ISO: tahun-bulan-hari	<code>2026-10-01T08:30:00</code>
UTF-8	Standar penyimpanan huruf; utf-8 aman untuk bahasa Indonesia & emoji	<code>open(f, encoding="utf-8")</code>
\n \t \	Baris baru, tab, satu backslash dalam teks	<code>"a\nb" -> dua baris</code>
%d %s %f	Penanda format gaya lama: angka bulat, teks, desimal	<code>"Umur %d" % 16</code>
{0} {1}	Penanda urutan di <code>.format()</code> / C#	<code>"{0} dan {1}".format("a", "b")</code>

8. JSON, .env, JavaScript, HTML, Bash

8.1 JSON - format tukar data

JSON dipakai hampir di mana-mana: API, file pengaturan, `package.json`. Aturannya ketat:

```
{
  "nama": "kelastrack",
  "versi": "1.2.0",
  "aktif": true,
  "port": 8081,
  "admin": null,
  "tag": ["sekolah", "absensi"],
  "database": {
    "host": "localhost",
    "port": 5432
  }
}
```

Simbol / bentuk	Arti
{ }	Objek: kumpulan pasangan "kunci": nilai
[]	Daftar (array) nilai
"nama": "x"	Kunci selalu dalam kutip dua, titik dua memisahkan kunci dan nilai
,	Pemisah item. Tidak boleh ada koma setelah item terakhir (error umum!)
true false null	Huruf kecil semua (beda dengan Python: True, False, None)
Kutip satu ' '	Tidak sah di JSON; harus kutip dua
Komentar # atau //	Tidak diizinkan di JSON

8.2 File .env - pengaturan rahasia

```
# .env - pengaturan rahasia/lingkungan (JANGAN di-upload ke GitHub)
DATABASE_URL=postgresql://user:password@localhost:5432/kelastrack
SECRET_KEY=isi-dengan-teks-acak-panjang
PORT=8081
DEBUG=true
```

Format **NAMA=nilai**, satu per baris, tanpa spasi di sekitar **=**. Isinya sering memuat password & kunci rahasia, jadi **jangan pernah di-upload ke GitHub** (masukkan `.env` ke file `.gitignore`).

8.3 JavaScript (Node.js)

Bahasa untuk web (di browser) dan juga server (Node.js). File `.js`, dijalankan `node nama.js`.

```
// contoh.js - JavaScript dasar (jalankan: node contoh.js)
const siswa = [
  { nama: "Budi", nilai: 80 },
  { nama: "Sari", nilai: 95 },
  { nama: "Andi", nilai: 60 },
];

// arrow function: (parameter) => hasil
const lulus = siswa.filter((s) => s.nilai >= 75);

for (const s of lulus) {
  console.log(`${s.nama} lulus dengan nilai ${s.nilai}`);
}

const rata = siswa.reduce((total, s) => total + s.nilai, 0) / siswa.length;
console.log("Rata-rata:", rata.toFixed(1));
```

Simbol	Arti	Contoh
;	Akhir pernyataan (sering opsional)	let x = 5;
const / let	Buat variabel tetap / variabel bisa berubah	const pi = 3.14
=>	Panah: fungsi singkat (arrow function)	(a, b) => a + b
===	Sama persis (nilai dan tipe). Gunakan ini, bukan ==	5 === "5" -> false
!==	Tidak sama persis	a !== b
&& !	dan, atau, bukan	a > 1 && a < 9
`	Backtick: teks dengan <code>\${variabel}</code> di dalamnya	`Halo \${nama}`
? :	Operator ternary: kondisi ? jika benar : jika salah	umur > 17 ? "dewasa" : "anak"
...	Spread: bongkar isi array/objek	[...a, 4]
// /* */	Komentar satu baris / banyak baris	// catatan

npm - pemasang library JavaScript

```
node --version           # cek Node terpasang
npm install              # pasang semua library dari package.json
npm install axios        # pasang satu library
npm run dev              # jalankan skrip "dev" milik proyek (biasanya server pengembangan)
npm run build            # bangun versi final untuk dipublikasikan
npm start                # jalankan aplikasi
```

8.4 HTML + CSS + JavaScript dalam satu file

Simpan sebagai `halaman.html` lalu klik dua kali; langsung terbuka di browser.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Halaman Pertama</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 2rem; }
    .kartu { border: 1px solid #ccc; padding: 1rem; border-radius: 8px; }
    #tombol { background: #2f6db5; color: white; padding: 8px 14px; }
  </style>
</head>
<body>
  <div class="kartu">
    <h1 id="judul">Halo Dunia</h1>
    <button id="tombol">Klik saya</button>
  </div>
  <script>
    document.getElementById("tombol").addEventListener("click", () => {
      document.getElementById("judul").textContent = "Tombol diklik!";
    });
  </script>
</body>
</html>
```

`<tag>` pembuka dan `</tag>` penutup (garis miring = penutup). `.kartu` di CSS = elemen ber-`class="kartu"`; `#tombol` = elemen ber-`id="tombol"`. `{ }` di CSS membungkus aturan gaya, dan `;` mengakhiri tiap aturan.

8.5 Bash (Linux / Mac / Git Bash)

```
#!/bin/bash
# setup.sh - versi Linux/macOS dari setup.ps1
set -e                                # berhenti kalau ada error

for p in python3 node git; do
    if command -v "$p" > /dev/null; then
        echo "[OK]    $p ditemukan"
    else
        echo "[GAGAL] $p belum terpasang"
        exit 1
    fi
done

cd backend
[ -d .venv ] || python3 -m venv .venv  # buat venv kalau belum ada
source .venv/bin/activate             # aktifkan venv
pip install -r requirements.txt
echo "Selesai!"
```

Bedanya dengan PowerShell: variabel dipanggil **\$nama** tetapi diisi tanpa **\$** (**nama=5**, **tanpa spasi** sekitar **=**). Kondisi pakai **[...]**, blok **if** diakhiri **fi**, perulangan **for** diakhiri **done**.

9. Git: menyimpan versi & upload kode

Git mencatat riwayat perubahan kode. GitHub adalah tempat menyimpan (online) hasil Git. Perintah dijalankan di terminal, di dalam folder proyek.

```
git --version                # cek Git terpasang

# --- mulai ---
git clone https://github.com/user/proyek.git    # unduh proyek dari GitHub
cd proyek
git init                                         # atau: jadikan folder sendiri repo Git

# --- siklus harian ---
git status                                     # lihat file apa yang berubah
git add .                                       # tandai SEMUA perubahan (titik = folder ini)
git add main.py                                # atau tandai satu file saja
git commit -m "Tambah fitur absensi"          # simpan jadi satu 'versi' + pesan
git push                                       # kirim ke GitHub
git pull                                       # ambil perubahan terbaru dari GitHub

# --- cabang (branch) ---
git branch                                     # lihat daftar cabang
git checkout -b fitur-baru                     # buat & pindah ke cabang baru
git checkout main                              # kembali ke cabang main
git merge fitur-baru                           # gabungkan fitur-baru ke cabang sekarang

# --- lihat riwayat ---
git log --oneline                             # daftar commit ringkas
git diff                                       # lihat isi perubahan
```

Istilah	Arti
repository (repo)	Folder proyek yang dilacak Git.
commit	Satu titik simpan (versi) dengan pesan.
branch	Jalur kerja terpisah, supaya eksperimen tidak merusak versi utama.
origin	Nama bawaan untuk alamat repo di GitHub.
main / master	Nama cabang utama.
HEAD	Penunjuk 'posisi saya sekarang' di riwayat.
.gitignore	File berisi daftar file/folder yang TIDAK ikut dilacak (misal <code>.venv</code> , <code>node_modules</code> , <code>.env</code>).
conflict	Dua orang mengubah baris yang sama; harus diputuskan manual mana yang dipakai.

Isi `.gitignore` contoh:

```
.venv/
node_modules/
__pycache__/
*.pyc
.env
*.log
```

10. Sistem logika: cara berpikir seperti programmer

Sintaks bisa dicari, tetapi **cara berpikir** harus dilatih. Semua program, di bahasa apa pun, dibangun dari ide yang sama. Bab ini adalah 'otak' di balik semua simbol sebelumnya.

10.1 Kerangka dasar: Input - Proses - Output

INPUT (data masuk)	PROSES (langkah-langkah)	OUTPUT (hasil)
nilai: 80, 75, 90 nama file klik tombol	jumlahkan, bagi 3 buka, baca, hitung baris ambil data, ubah	rata-rata 81.67 "120 baris" halaman baru

Kalau bingung mulai dari mana, tulis tiga hal: **apa inputnya**, **apa outputnya yang diinginkan**, **apa langkah di antaranya?** Program hanyalah daftar langkah yang sangat teliti.

10.2 Tiga struktur dasar semua program

Struktur	Pertanyaan yang dijawab	Kata kunci	Perumpamaan
Urutan (sequence)	Langkah apa dulu, baru apa?	baris demi baris	Resep masak
Pilihan (selection)	Kalau begini lakukan A, kalau tidak lakukan B	if / elif / else	Persimpangan jalan
Perulangan (iteration)	Lakukan berulang untuk tiap item / sampai syarat	for, while	Menyetempel tiap lembar

Ditambah satu alat penyusun: **fungsi** (bungkus langkah supaya bisa dipakai ulang). Semua program, dari kalkulator sampai aplikasi absensi, adalah kombinasi empat hal ini.

```
# Contoh: hitung kelulusan beberapa siswa.      Ketiganya muncul sekaligus.
daftar_nilai = [80, 55, 92]                     # data
for nilai in daftar_nilai:                       # 3. PERULANGAN: untuk tiap nilai
    if nilai >= 60:                               # 2. PILIHAN: lulus atau tidak?
        print(nilai, "lulus")
    else:
        print(nilai, "tidak lulus")
print("selesai")                                # 1. URUTAN: baris ini jalan setelah loop selesai
```

Diagram alur (flowchart) dari pilihan di atas



Lambang: oval = mulai/selesai, persegi = proses, belah ketupat < > = pertanyaan ya/tidak. Menggambar alur di kertas sebelum menulis kode mencegah banyak kesalahan.

10.3 Logika Boolean: benar atau salah

Setiap pertanyaan **if** menghasilkan **True** (benar) atau **False** (salah). Tiga operator utamanya:

A	B	A and B (dan)	A or B (atau)	not A (bukan)	A xor B (salah satu saja)
True	True	True	True	False	False
True	False	False	True	False	True
False	True	False	True	True	True
False	False	False	False	True	False

- **AND**: semua syarat harus benar. 'Boleh masuk jika punya tiket **dan** membawa KTP.'
- **OR**: cukup satu yang benar. 'Gratis jika anak-anak **atau** lansia.'
- **NOT**: membalik. 'Bukan hari libur.'
- **Urutan pengerjaan**: **not** dulu, lalu **and**, lalu **or**. Gunakan **kurung** agar jelas: **(a or b) and c**.
- **Short-circuit**: pada **A and B**, kalau A sudah False maka B tidak diperiksa. Dipakai untuk menghindari error: **if x != 0 and 10 / x > 2:**.
- **Hukum De Morgan**: **not (A and B)** sama dengan **(not A) or (not B)**; **not (A or B)** sama dengan **(not A) and (not B)**.

Apa yang dianggap benar atau salah (truthy / falsy)

Bahasa	Dianggap SALAH	Dianggap BENAR
Python	<code>False, None, 0, 0.0, "", [], {}</code>	Selain itu, termasuk <code>"0"</code> dan <code>"False"</code> (teks tak kosong)
JavaScript	<code>false, null, undefined, 0, NaN, ""</code>	Selain itu, termasuk <code>[]</code> dan <code>{}</code> (kosong pun benar!)
PowerShell	<code>\$false, \$null, 0, "", array kosong</code>	Selain itu, termasuk <code>"0"</code> dan <code>"False"</code>

10.4 Variabel dan 'jejak' (dry run)

Variabel itu kotak yang isinya bisa berubah. Cara terbaik memahami kode (atau mencari bug) adalah **melacak isi kotak baris demi baris di kertas**, tanpa menjalankan komputer. Ini disebut *dry run* atau tabel jejak.

```
total = 0
for angka in [3, 5, 2]:
    total = total + angka
print(total)
```

Langkah	angka	total sebelum	total sesudah	Catatan
awal	-	-	0	<code>total = 0</code>
putaran 1	3	0	3	<code>0 + 3</code>
putaran 2	5	3	8	<code>3 + 5</code>
putaran 3	2	8	10	<code>8 + 2</code>
selesai	-	-	10	<code>print(total)</code> mencetak 10

Kalau hasil program tidak sesuai harapan, buat tabel seperti ini dengan data kecil. Baris tempat isi variabel berbeda dari dugaan lu adalah letak bug.

10.5 Memecah masalah besar jadi kecil (dekomposisi)

Masalah besar ('buat aplikasi absensi') tidak dikerjakan sekaligus. Pecah sampai tiap potongan bisa dikerjakan dalam 5 menit:

```
Aplikasi absensi
|-- 1. Simpan data siswa          -> fungsi tambah_siswa(nama, kelas)
|-- 2. Catat kehadiran per hari   -> fungsi catat(nama, tanggal, status)
|-- 3. Hitung persen kehadiran    -> fungsi persen_hadir(daftar)
|-- 4. Putuskan boleh ujian atau tidak -> fungsi boleh_ujian(...)
'-- 5. Tampilkan laporan         -> fungsi cetak_laporan()
```

Kerjakan dan **tes satu fungsi sampai benar** sebelum lanjut ke yang berikutnya. Ini juga alasan fungsi dibuat kecil dan bernama jelas.

10.6 Enam langkah menyelesaikan soal / masalah

Langkah	Lakukan	Contoh (hitung rata-rata)
1. Pahami	Tulis ulang masalah dengan kata sendiri.	'Ambil beberapa angka, hasilkan nilai tengahnya.'
2. Contoh	Tulis 2-3 contoh input dan output yang benar dengan tangan .	[80,70,90] -> 80; [100] -> 100; [] -> ??? (pertanyaan baru!)
3. Rencana	Tulis langkah dalam bahasa biasa (pseudocode).	jumlahkan semua; bagi dengan banyaknya
4. Kode kecil	Terjemahkan jadi kode, satu bagian dulu.	<code>sum(daftar) / len(daftar)</code>
5. Tes	Coba contoh normal, kasus batas (kosong, 1 item, 0, negatif), dan kasus salah.	Daftar kosong -> bagi nol! Tambahkan pemeriksaan.
6. Rapikan	Beri nama jelas, hapus yang berulang, tambah komentar secukupnya.	Pindahkan jadi fungsi <code>rata_rata()</code>

10.7 Kesalahan logika yang paling sering

Kesalahan	Contoh	Cara menghindari
= vs ==	<code>if x = 5</code> (mengisi) padahal maksudnya <code>if x == 5</code> (membandingkan)	Ingat: satu sama dengan = mengisi, dua = bertanya.
Selisih satu (off-by-one)	<code>range(1, 5)</code> hanya 1-4, bukan 1-5. Item terakhir daftar berindeks <code>len-1</code> .	Ingat: indeks mulai 0; akhir <code>range</code> tidak ikut.
Urutan <code>if</code> salah	Cek <code>nilai >= 60</code> dulu lalu <code>nilai >= 90</code> : nilai 95 tidak pernah sampai 'A'.	Letakkan kondisi paling spesifik/ketat di atas.
Loop tak pernah berhenti	<code>while x < 10</code> : tetapi <code>x</code> tidak pernah bertambah.	Pastikan ada baris yang mengubah syarat; tekan Ctrl+C .
Teks vs angka	<code>"5" + "3" = "53"</code> , bukan 8. Hasil <code>input()</code> selalu teks.	Ubah dengan <code>int()</code> / <code>float()</code> .
Pembagian bulat	<code>7 // 2 = 3</code> (bukan 3.5).	Pakai <code>/</code> untuk hasil desimal.
Desimal tidak tepat	<code>0.1 + 0.2 = 0.30000000000000004</code>	Bandingkan dengan <code>round()</code> atau toleransi, jangan <code>==</code> .
Besar-kecil huruf	<code>"Budi" == "budi"</code> salah.	<code>.lower()</code> pada kedua sisi.
Mengubah daftar saat di looping	Menghapus item dari list ketika <code>for</code> sedang membacanya.	Buat daftar baru berisi hasil.
Lupa kasus kosong / None	Membaca <code>daftar[0]</code> saat daftar kosong -> <code>IndexError</code> .	Periksa <code>if daftar:</code> dulu.
Zona waktu / format tanggal	'2026-10-01' vs '01/10/2026' berbeda tafsir.	Pakai format ISO <code>YYYY-MM-DD</code> di dalam program.

10.8 Studi kasus lengkap: boleh ikut ujian atau tidak?

Aturan sekolah: siswa boleh ikut ujian jika (kehadiran minimal 75% **dan** alfa maksimal 3 kali) **atau** memiliki surat dispensasi.

Langkah 1 - ubah aturan jadi huruf: A = kehadiran $\geq 75\%$, B = alfa ≤ 3 , C = punya dispensasi. Rumus: **boleh = (A and B) or C**.

Langkah 2 - tabel kebenaran (periksa semua kemungkinan):

A ($\geq 75\%$)	B (alfa ≤ 3)	C (dispensasi)	A and B	Boleh ujian?
Ya	Ya	Ya	Ya	BOLEH
Ya	Ya	Tidak	Ya	BOLEH
Ya	Tidak	Ya	Tidak	BOLEH (dispensasi)
Ya	Tidak	Tidak	Tidak	TIDAK
Tidak	Ya	Ya	Tidak	BOLEH (dispensasi)
Tidak	Ya	Tidak	Tidak	TIDAK
Tidak	Tidak	Ya	Tidak	BOLEH (dispensasi)
Tidak	Tidak	Tidak	Tidak	TIDAK

Langkah 3 - diagram alur:

```

< punya dispensasi? > --ya--> BOLEH
      |
      tidak
      v
< kehadiran >= 75% DAN alfa <= 3 ? > --ya--> BOLEH

```

|
tidak
v
TIDAK

Langkah 4 - kode Python:

```
# absensi.py - logika: siapa yang boleh ikut ujian?
# H = hadir, S = sakit, I = izin, A = alfa (tanpa keterangan)
catatan = {
    "Budi": ["H", "H", "H", "S", "H", "H", "I", "H"],
    "Sari": ["H", "H", "H", "H", "H", "H", "H", "H"],
    "Andi": ["H", "A", "A", "A", "A", "H", "H", "H"],
}
dispensasi = {"Andi"} # siswa yang punya surat dispensasi

def persen_hadir(daftar):
    return daftar.count("H") / len(daftar) * 100

def boleh_ujian(nama, daftar):
    persen = persen_hadir(daftar)
    alfa = daftar.count("A")
    memenuhi_syarat = persen >= 75 and alfa <= 3 # dua syarat harus benar
    return memenuhi_syarat or nama in dispensasi # atau punya dispensasi

for nama, daftar in catatan.items():
    ok = boleh_ujian(nama, daftar)
    print(f"{nama:<5} hadir {persen_hadir(daftar):5.1f}% alfa {daftar.count('A')} ->",
          "BOLEH" if ok else "TIDAK")
```

Hasil yang muncul:

```
Budi  hadir  75.0%  alfa 0  -> BOLEH
Sari  hadir 100.0%  alfa 0  -> BOLEH
Andi  hadir  50.0%  alfa 4  -> BOLEH
```

Andi hanya lolos karena **dispensasi**. Hapus Andi dari himpunan (**dispensasi = set()**) dan hasilnya berubah menjadi **TIDAK**. Budi persis 75%: itu alasan memakai **>=** bukan **>**, contoh nyata kesalahan 'selisih satu' yang harus diuji.

Kasus uji yang perlu dicoba

Uji	Data	Harapan
Batas tepat	6 hadir dari 8 (75%), alfa 0	BOLEH
Hampir batas	5 hadir dari 8 (62.5%)	TIDAK
Batas alfa	alfa 3 vs alfa 4 (persen cukup)	3 -> BOLEH, 4 -> TIDAK
Kosong	daftar kehadiran <code>[]</code>	Harus ditangani: sekarang akan error bagi nol. Perbaiki!
Dispensasi	kehadiran 0% + dispensasi	BOLEH

10.9 Cara mencari bug (metode ilmiah)

- 1. **Reproduksi:** temukan langkah pasti yang memunculkan masalah. Bug yang tak bisa diulang tak bisa diperbaiki.
- 2. **Baca pesan error dari bawah** (bab 11), catat file + nomor barisnya.
- 3. **Persempit:** bagi dua. Beri `print()` di tengah program: salah sebelum atau sesudah titik itu? Ulangi sampai ketemu baris pelakunya.
- 4. **Buat dugaan:** 'mungkin variabel `x` kosong di sini'. Cetak nilainya: `print("x =", x)`.
- 5. **Ubah SATU hal** lalu uji. Mengubah banyak hal sekaligus membuat lu tak tahu mana yang menolong.
- 6. **Lihat apa yang berubah:** `git diff` menunjukkan perbedaan dari versi yang dulu masih jalan.

- **7. Jelaskan pada 'bebek karet':** menjelaskan kode baris per baris dengan suara keras sering langsung membuka kesalahannya.
- **8. Cari:** salin kalimat error tanpa nama file/jalur pribadi ke mesin pencari.

Cara bertanya

Format bertanya yang membuat orang (atau AI) cepat menolong:

(1) apa yang lu coba lakukan, (2) apa yang diharapkan, (3) apa yang terjadi, (4) **pesan error lengkap** (teks, bukan foto), (5) versi OS / Python / Node, (6) potongan kode terkait yang paling kecil.

10.10 Alur aplikasi web (dan kenapa angka port, JSON, dan API ada)

BROWSER (client)		SERVER		DATABASE
+-----+	request	+-----+	query	+-----+
halaman web	----->	API (FastAPI,	----->	MongoDB /
HTML/CSS/JS	GET /api/..	Node, dll)		PostgreSQL
	<-----	port 8081	<-----	
+-----+	response	+-----+	hasil	+-----+
	200 + JSON			

Satu klik tombol = satu atau lebih siklus **request - response**. Request punya *metode* (apa yang diinginkan) dan *alamat*; response punya *kode status* (angka, bab 7.7) dan *isi* (biasanya JSON).

Tujuan (CRUD)	Metode HTTP	Contoh alamat	SQL	MongoDB
Create: buat data	POST	/api/siswa	INSERT	insertOne
Read: baca data	GET	/api/siswa/5	SELECT	find
Update: ubah data	PUT / PATCH	/api/siswa/5	UPDATE	updateOne
Delete: hapus data	DELETE	/api/siswa/5	DELETE	deleteOne

Memanggil API dari terminal (untuk mencoba tanpa browser):

```
# PowerShell
Invoke-RestMethod http://localhost:8081/api/settings
Invoke-RestMethod -Method Post -Uri http://localhost:8081/api/siswa `
  -ContentType "application/json" -Body '{"nama":"Budi","kelas":"10A"}'

# CMD / Linux / Mac / Git Bash
curl http://localhost:8081/api/settings
curl -X POST http://localhost:8081/api/siswa \
  -H "Content-Type: application/json" \
  -d '{"nama":"Budi"}'           # (di CMD pakai kutip dua dan escape: "{\"nama\":\"Budi\"}")
```

11. Membaca pesan error dan log

Error bukan tanda gagal, itu petunjuk. Cara membacanya:

- Baca dari **baris paling bawah**: itu jenis & penyebab error.
- Cari **nomor baris** dan **nama file** (misal **File "main.py", line 12**) - di situ letak masalahnya, atau tepat di atasnya.
- Salin kalimat error ke Google / tanya AI - hampir semua error sudah pernah dialami orang lain.
- Ubah **satu hal** per percobaan, lalu jalankan lagi.

```
Traceback (most recent call last):  
  File "main.py", line 12, in <module>          <- di file & baris mana  
    hasil = hitung(5)  
  File "main.py", line 7, in hitung  
    return x / y  
NameError: name 'y' is not defined              <- BACA INI: variabel 'y' belum dibuat
```

11.1 Error umum & solusinya

Pesan error	Biasanya karena	Solusi
'python' is not recognized... / The term 'python' is not recognized	Program belum terpasang atau tidak ada di PATH	Pasang ulang; saat install Python centang Add to PATH . Tutup & buka lagi terminal.
running scripts is disabled on this system	ExecutionPolicy PowerShell	<code>Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass</code>
Cannot find path ... because it does not exist	Salah folder atau salah ketik	<code>Get-Location</code> & <code>Get-ChildItem</code> untuk cek; pakai Tab.
The term '.\setup.ps1' is not recognized	File tidak ada di folder ini	<code>cd</code> ke folder yang benar.
IndentationError	Spasi/jorokan Python salah	Rapikan jadi 4 spasi konsisten.
SyntaxError: invalid syntax	Salah tulis: kurang :, kurung, atau kutip	Lihat baris yang ditunjuk dan baris di atasnya.
NameError: name 'x' is not defined	Variabel belum dibuat / salah ketik / beda huruf besar-kecil	Cek ejaan; Python membedakan Nama dan nama .
TypeError: unsupported operand ... 'int' and 'str'	Mencampur angka dan teks	<code>int(teks)</code> atau <code>str(angka)</code> .
IndexError: list index out of range	Indeks melebihi jumlah item	Ingat indeks mulai 0; terakhir = <code>len(a)-1</code> .
KeyError: 'nama'	Kunci dictionary tidak ada	Pakai <code>d.get("nama")</code> atau cek ejaan.
ModuleNotFoundError: No module named 'fastapi'	Library belum dipasang, atau venv tidak aktif	Aktifkan <code>.venv</code> , lalu <code>pip install fastapi</code> .
FileNotFoundError	Jalur file salah / kerja di folder lain	Cek folder sekarang; pakai jalur lengkap.
UnicodeDecodeError / huruf aneh	Encoding file tidak cocok	<code>open(f, encoding="utf-8")</code> .
Address already in use / port in use	Port sudah dipakai program lain	Tutup program lain (Ctrl+C) atau ganti port.
Connection refused / ECONNREFUSED	Server tujuan belum jalan	Jalankan server dulu; cek nomor port.
npm ERR! / command not found: npm	Node.js belum terpasang atau salah folder	Pasang Node.js; jalankan di folder yang ada <code>package.json</code> .
Permission denied / Access is denied	Tidak punya izin	Jalankan terminal sebagai Administrator, atau cek izin file.
fatal: not a git repository	Bukan di dalam folder Git	<code>cd</code> ke folder proyek atau <code>git init</code> .
JSONDecodeError	JSON tidak valid	Cek koma berlebih, kutip satu, komentar.
Terminal diam, tidak ada cursor/prompt	Ada kutip/kurung yang belum ditutup, atau program masih berjalan	Tekan Ctrl+C .

11.2 Membaca log: cara umum (5 pertanyaan)

Log adalah 'buku harian' program, satu kejadian per baris. Jangan membaca semuanya. Cari jawaban 5 pertanyaan ini pada satu baris log:

Pertanyaan	Cari di	Contoh
Kapan?	stempel waktu (timestamp), biasanya paling kiri	<code>2026-10-01T08:15:30.123+07:00</code>

Pertanyaan	Cari di	Contoh
Seberapa parah?	level: DEBUG / INFO / WARN / ERROR / FATAL (atau huruf D I W E F)	"s": "W", ERROR
Siapa yang bicara?	komponen / modul / thread / nama logger	"c": "NETWORK", uvicorn.error
Apa yang terjadi?	pesan (msg / message)	"msg": "Connection accepted"
Detailnya?	data tambahan (attr / extra) atau tumpukan error (stack trace)	"attr": {...}

Aturan emas: log penuh baris DEBUG/INFO itu normal dan tidak berbahaya. Fokus pada **WARN**, **ERROR**, **FATAL**, dan baris yang berisi kata seperti *error*, *exception*, *failed*, *refused*, *denied*, *timeout*, *corrupt*, *out of memory*, *no space left*.

Dua log web yang sering dijumpai

```
INFO:      127.0.0.1:52344 - "GET /api/settings HTTP/1.1" 200 OK
|           |           |           |           |           | |
| level      | alamat peminta | metode | jalur      | protokol | status |
|           |           |           |           |           |           |
127.0.0.1 - - [01/Oct/2026:08:00:00 +0700] "GET /api/settings HTTP/1.1" 200 512
|           |           |           |           |           |
| peminta    | waktu          | metode+jalur |           | status | ukuran balasan (byte) |
```

Yang paling penting: kode status (200 = sukses, 404 = tak ditemukan, 500 = bug di server; lihat 7.7) dan jalurnya.

11.3 Contoh nyata: log MongoDB (WiredTiger) WT_SESSION.checkpoint

Potongan yang lu tanyakan:

```
WT_SESSION.checkpoint", "category": "WT_VERB_CHECKPOINT_PROGRESS", "category_id": 6,
"verbose_level": "DEBUG_1", "verbose_level_id": 1,
"msg": "saving checkpoint snapshot min: 215, snapshot max: 215 snapshot count: 0,
oldest timestamp: (0, 0) , meta checkpoint timestamp: (0, 0) base write gen: 20817"}}
```

(Di atas dipecah jadi beberapa baris supaya muat di halaman; aslinya satu baris panjang.)

Ini adalah **satu baris log MongoDB** (format *JSON per baris*, tiap baris adalah satu objek JSON utuh). Potongan lu dimulai di tengah baris. Akhiran **}}}** (tiga kurung kurawal tutup) menutup tiga lapis objek JSON: objek **message**, objek **attr**, dan objek terluar. Bentuk lengkapnya kira-kira seperti ini (dirapikan; selain isi yang lu tempel, nilai lain hanya contoh ilustrasi):

```
{
  "t": { "$date": "2026-10-01T08:15:30.123+07:00" }, <- kapan
  "s": "D1", <- tingkat: Debug level 1
  "c": "WTCHKPT", <- komponen: WiredTiger Checkpoint
  "id": 22430, <- nomor jenis pesan
  "ctx": "Checkpointner", <- thread yang menulis
  "msg": "WiredTiger message", <- judul pesan
  "attr": { <- data rinci
    "message": {
      "ts_sec": 1790000130,
      "ts_usec": 123456,
      "thread": "1234:1400000000000000",
      "session_name": "WT_SESSION.checkpoint", <- sesi: pembuatan checkpoint
      "category": "WT_VERB_CHECKPOINT_PROGRESS", <- kategori: kemajuan checkpoint
      "category_id": 6,
      "verbose_level": "DEBUG_1", <- tingkat kerincian: debug
      "verbose_level_id": 1,
      "msg": "saving checkpoint snapshot min: 215, snapshot max: 215 snapshot count: 0, ..."
    }
  }
}
```

Bagian	Arti
WT_SESSION.checkpoint	Operasi checkpoint di mesin penyimpanan WiredTiger (mesin penyimpanan bawaan MongoDB). Checkpoint = menyimpan keadaan data yang ada di memori ke disk secara berkala (biasanya sekitar tiap 60 detik) supaya data aman bila komputer mati.
WT_VERB_CHECKPOINT_PROGRESS	Kategori log: laporan kemajuan checkpoint.
DEBUG_1 / s: D1	Tingkat debug paling rendah: catatan rinci untuk teknisi, bukan error .
snapshot min: 215, snapshot max: 215, snapshot count: 0	Rentang nomor transaksi yang terlihat saat checkpoint, dan jumlah transaksi yang sedang berjalan (0 = tidak ada yang menahan).
oldest timestamp: (0, 0), meta checkpoint timestamp: (0, 0)	Penanda waktu internal; (0, 0) artinya belum ada/tidak dipakai.
base write gen: 20817	Nomor 'generasi' penulisan internal. Angka internal, tidak perlu dihafal.

Jadi: perlu panik?

Kesimpulan: baris ini **normal dan aman**. Tidak ada kata error/warning, levelnya DEBUG. MongoDB sedang rutin menyimpan checkpoint.

Mengurangi kebisingan log

Kalau log kebanyakan baris DEBUG seperti ini, tingkat detail log MongoDB kemungkinan diset tinggi (opsi **systemLog.verbosity** di **mongod.conf**, atau perintah **db.setLogLevel**). Nilai **0** = log normal. Tanda masalah sungguhan di log MongoDB: nilai **"s"** berupa **"W"**, **"E"**, atau **"F"**; atau **msg** berisi **error**, **failed**, **exception**, **corrupt**, **No space left on device**, **Address already in use**.

11.4 Menyaring log dari terminal

```
# --- PowerShell ---
Get-Content .\mongod.log -Tail 50 # 50 baris terakhir
Get-Content .\mongod.log -Wait -Tail 10 # ikuti log langsung (Ctrl+C berhenti)
Select-String '"s":"E"' .\mongod.log # hanya baris ERROR
Get-Content .\mongod.log | Where-Object { $_ -match '"s":"(E|F|W)"' } # error + fatal + warning
Select-String -Pattern "error|failed|refused" -Path .\mongod.log # kata kunci

# satu baris -> objek -> ambil bagian tertentu
Get-Content .\mongod.log -Tail 1 | ConvertFrom-Json | Select-Object -ExpandProperty msg

# rapikan satu baris JSON panjang agar mudah dibaca
Get-Content .\mongod.log -Tail 1 | ConvertFrom-Json | ConvertTo-Json -Depth 10

# --- Linux / Mac / Git Bash ---
tail -n 50 mongod.log
tail -f mongod.log
grep '"s":"E"' mongod.log
tail -n 1 mongod.log | python -m json.tool # rapikan JSON
```

Cara paling mudah di VS Code: buka file log, tekan **Ctrl+F**, ketik **"s":"E"** atau kata **error**, lalu Enter untuk loncat antar kemunculan. Satu baris JSON panjang bisa dirapikan dengan menyalinnya ke file **.json** baru lalu **Shift+Alt+F**.

Menyaring log JSON dengan Python (sudah diuji pada contoh log buatan):

```
# cari_error.py - tampilkan hanya baris Warning/Error/Fatal dari log JSON
import json

with open("mongod.log", encoding="utf-8") as f:
    for no, baris in enumerate(f, 1):
        try:
            d = json.loads(baris) # ubah teks JSON jadi dictionary
        except json.JSONDecodeError: # lewati baris yang bukan JSON
            continue
        if d.get("s") in ("E", "F", "W"):
```

```
print(no, d["t"]["$date"], d["s"], d["c"], d["msg"])
```

Tingkat keparahan log MongoDB

Huruf (🔴)	Arti	Perlu bertindak?
F	Fatal: server berhenti	Ya, segera
E	Error	Ya
W	Warning	Baca, mungkin
I	Info normal	Tidak
D1 ... D5	Debug, makin besar angkanya makin rinci	Tidak

12. Bedah perintah nyata (proyek KelasTrack)

12.1 Hapus folder .venv

```
Remove-Item ".\app-clastrack-main\backend\.venv" -Recurse -Force -ErrorAction SilentlyContinue
```

Bagian	Arti
Remove-Item	Perintah hapus (Verb-Noun: Remove + Item).
".\app-clastrack-main\backend\.venv"	Alamat target. <code>.</code> = mulai dari folder ini; tiap <code>\</code> pemisah folder; <code>.venv</code> nama folder (titik di depan = folder 'tersembunyi'). Dikutip karena aman jika ada spasi.
-Recurse	Hapus juga seluruh isi di dalamnya.
-Force	Paksa; tanpa bertanya, termasuk file tersembunyi / read-only.
-ErrorAction SilentlyContinue	Kalau folder tidak ada, diam saja (tidak tampil error merah).

12.2 Membuat ulang venv & memasang library

```
cd .\app-clastrack-main\backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Bagian	Arti
cd .\app-clastrack-main\backend	Masuk ke folder backend (relatif dari folder sekarang).
python -m venv .venv	<code>-m</code> = jalankan modul bawaan Python bernama <code>venv</code> ; hasilnya folder <code>.venv</code> .
.\.venv\Scripts\Activate.ps1	Jalankan script pengaktif (<code>.</code> folder ini, <code>.venv\Scripts</code> lokasi, <code>Activate.ps1</code> filenya).
pip install -r requirements.txt	<code>-r</code> = baca daftar library dari file <code>requirements.txt</code> .

12.3 Menjalankan beberapa perintah sekaligus

```
cd backend; python -m venv .venv; .\.venv\Scripts\Activate.ps1; pip install -r requirements.txt
```

Titik koma `;` = 'yang ini selesai, lanjut yang itu'. Di PowerShell terbaru ada juga `&&` (lanjut hanya jika sebelumnya sukses) dan `||` (lanjut hanya jika gagal).

```
python -m venv .venv && echo "venv berhasil"           # echo hanya jika berhasil
python -m venv .venv || echo "venv gagal"              # echo hanya jika gagal
```

12.4 Menyimpan hasil & menyaring

```
Get-ChildItem -Recurse | Where-Object { $_.Length -gt 1MB } | Select-Object Name, Length
```

Dibaca dari kiri: ambil semua file termasuk subfolder (`-Recurse`) -> **pipa** | -> saring yang ukurannya (`$_Length`) lebih besar (`-gt`) dari 1MB -> **pipa** -> tampilkan hanya kolom Name dan Length.

```
Get-Content .\log.txt -Tail 20           # 20 baris terakhir
Select-String "ERROR" .\log.txt          # cari kata ERROR di file
Get-Process | Where-Object { $_.ProcessName -like "python*" } # proses python yang jalan
Stop-Process -Name python -Force         # matikan semua proses python
Invoke-WebRequest http://localhost:8081/api/settings          # coba panggil API
```

13. Cheat sheet satu halaman

Simbol ringkas

Simbol	Arti singkat	Simbol	Arti singkat
\	pemisah folder Windows	/	pemisah folder web/Linux; bagi
.	folder ini / ambil isi objek	..	folder di atas
" "	teks (variabel diganti di PS)	' '	teks apa adanya
[]	list/array/indeks	{ }	blok (PS/JS) / dict (Python)
()	panggil fungsi / kelompok	:	pasangan / penutup if (Python)
\$	variabel (PS/Bash)	#	komentar (PS/Python/Bash)
	pipa: kirim hasil ke kanan	;	pemisah perintah
-	awalan opsi / operator PS	&	jalankan (PS) / jalan di latar (Bash)
=	mengisi variabel	==	membandingkan (Python/JS)
> >>	simpan hasil ke file (timpa/tambah)	*	wildcard / kali
~	folder home	!	tidak (not)
%	sisa bagi	@	dekorator (Py), array/hash (PS)
=>	arrow function (JS)	->	tipe keluaran (Py) / 'menjadi' di buku ini
_	privat / diabaikan	__x__	nama khusus Python

Operator berdampingan

Fungsi	PowerShell	Python	JavaScript	Bash
sama dengan	-eq	==	===	-eq (angka) / = (teks)
tidak sama	-ne	!=	!==	-ne
lebih besar	-gt	>	>	-gt
dan	-and	and	&&	&&
atau	-or	or		
bukan	-not / !	not	!	!
komentar	#	#	//	#
kosong	\$null	None	null	(string kosong)
benar/salah	\$true \$false	True False	true false	true false
cetak	Write-Host	print()	console.log()	echo
variabel	\$x = 5	x = 5	let x = 5	x=5
if	if (c) { }	if c:	if (c) { }	if [c]; then ... fi
loop	foreach (\$i in \$a) { }	for i in a:	for (const i of a) { }	for i in a; do ... done

Lima kebiasaan yang mencegah 80% masalah

- Selalu cek dulu **lu ada di folder mana** (**pwd** / **Get-Location**).

- Pakai tombol **Tab** untuk melengkapi nama - jangan ketik panjang manual.
- Apit alamat berspasi dengan **tanda kutip**.
- Baca pesan error dari **baris paling bawah**, salin ke pencarian.
- Pakai **Ctrl+C** bila terminal menggantung; **panah atas** untuk mengulang perintah.
- Jangan upload **.env**, password, atau **.venv** / **node_modules** ke GitHub.

Saran terakhir

Belajar paling cepat: buka terminal, ketik tiap contoh di panduan ini sendiri, lalu **sengaja ubah** (ganti angka, hapus satu tanda kutip) dan lihat error apa yang muncul. Dari error itu lu belajar fungsi tiap simbol.

Lampiran A. Semua simbol keyboard

Seluruh tanda baca pada keyboard (layout US, yang paling umum untuk coding) beserta kegunaannya. Kolom 'Cara ketik' memakai **Shift + tombol**.

Simbol	Nama	Ketik	Kegunaan utama
`	backtick / grave	tombol kiri atas, di bawah Esc	PowerShell: escape & lanjut baris. JavaScript: template string. Markdown: kode. Bash: cara lama menjalankan perintah di dalam perintah (kini <code>\$(perintah)</code>).
~	tilde	Shift + `	Folder home (~), PowerShell sub-ekspresi; Bash: ~/proyek. ~~~ penanda salah di error PS.
!	tanda seru	Shift + 1	NOT (!x), tidak sama (!=), #! shebang, Bash: history.
@	at	Shift + 2	Python decorator; PowerShell array/hash (@(), @{}); email; CMD @echo off.
#	pagar / hash	Shift + 3	Komentar (Python, PowerShell, Bash); #id di CSS; #fragment di URL; heading di Markdown.
\$	dolar	Shift + 4	Variabel (PowerShell, Bash); akhir baris di regex; template JS \${x}.
%	persen	Shift + 5	Sisa bagi; CMD %VAR%; Python format %d; URL %20 = spasi.
^	caret	Shift + 6	Pangkat (beberapa bahasa), awal baris di regex, escape di CMD, ^ di versi npm.
&	ampersand	Shift + 7	AND bit; && dan; PowerShell: jalankan (& cmd); Bash: latar belakang; URL: pemisah parameter.
*	bintang	Shift + 8	Kali; wildcard *.txt; *args Python; regex 'nol atau lebih'; Markdown tebal.
()	kurung biasa	Shift + 9 / 0	Memanggil fungsi, mengelompokkan, tuple.
-	strip / minus	tombol 0 kanan	Kurang; awalan opsi (-Force, -r); ->; nama dengan strip.
_	garis bawah	Shift + -	Pemisah kata (nama_file); privat di Python; __init__.
=	sama dengan	tombol kanan strip	Mengisi variabel; == banding; => arrow JS; URL ?a=1.
+	plus	Shift + =	Tambah; gabung teks; +=; regex 'satu atau lebih'.
[]	kurung siku	tombol kanan P	List/array, indeks a[0], tipe [int] (PS), [] kondisi Bash, kelas karakter regex.
{ }	kurung kurawal	Shift + []	Blok kode (PS, JS, Bash fungsi), dict/set (Python), objek JSON, \${}, {{ }} template.
\	backslash	di atas Enter	Pemisah folder Windows; escape (\n, \t, \"); lanjut baris Bash; regex \d.
	pipa / garis tegak	Shift + \	Pipa (kirim hasil ke perintah berikut); atau; regex 'atau'; \ di Markdown tabel.
;	titik koma	kanan L	Pemisah perintah; akhir pernyataan JS/C; PATH Windows.
:	titik dua	Shift + ;	Akhir if/for/def Python; pasangan JSON; C:: \$env::; PATH Linux; URL http:.
'	kutip satu	kanan ;	Teks apa adanya (PS, Bash); teks Python/JS.
"	kutip dua	Shift + '	Teks; di PS/Bash variabel diganti nilainya; kunci JSON wajib kutip dua.
,	koma	kanan M	Pemisah item, argumen.

Simbol	Nama	Ketik	Kegunaan utama
< >	lebih kecil / lebih besar	Shift + , / .	Banding; tag HTML <div>; redirect >, >>, <; generik <code>List<int></code> .
.	titik	kanan koma	Folder ini (.\), .. folder atas, akses properti <code>a.b</code> , ekstensi file, regex 'karakter apa saja'.
/	slash	kanan titik	Bagi; pemisah folder web & Linux; URL; komentar <code>//</code> ; regex <code>/pola/</code> .
?	tanda tanya	Shift + /	Wildcard 1 karakter; regex 'opsional'; ternary <code>a ? b : c</code> ; URL <code>?q=cari</code> ; <code>?. JS</code> .

Lampiran B. Simbol gabungan dan karakter escape

Gabungan	Arti	Dipakai di
<code>== != <= >=</code>	sama, tidak sama, kurang/sama, lebih/sama	Python, JS, Bash <code>[[]]</code>
<code>=== !==</code>	sama/tidak sama persis (nilai dan tipe)	JavaScript
<code>&& </code>	dan / atau (juga: sambung perintah jika sukses/gagal)	JS, Bash, CMD, PowerShell 7
<code>++ --</code>	tambah / kurang 1	JS, PowerShell, C
<code>+= -= *= /= %=</code>	operasi lalu isi ke diri sendiri	hampir semua bahasa
<code>//</code>	bagi bulat (Python) / komentar (JS)	Python, JS
<code>**</code>	pangkat	Python, JS, Bash
<code>-> =></code>	tipe keluaran / fungsi panah	Python / JS, C#, PHP
<code>::</code>	pemisah namespace	C++, PowerShell <code>[Math]::Round()</code>
<code>..</code>	folder induk; rentang <code>1..5</code>	path; PowerShell, Bash
<code>...</code>	spread / sisa	JS, Python (<code>Ellipsis</code>)
<code>?. ??</code>	optional chaining, nilai cadangan	JS, C#, PowerShell 7
<code><< >></code>	geser bit / redirect heredoc / tambah ke file	C, Bash, CMD
<code>2>&1</code>	gabung error ke output normal	Bash, CMD, PowerShell
<code>\${x} \$(x) \$((x))</code>	isi variabel / hasil perintah / hitung	JS template, Bash, PowerShell
<code>%VAR%</code>	variabel lingkungan	CMD
<code>@{ } @()</code>	hashtable / array	PowerShell
<code>/* ... */</code>	komentar banyak baris	JS, CSS, C, Java
<code><!-- ... --></code>	komentar HTML	HTML
<code><# ... #></code>	komentar banyak baris PowerShell	PowerShell
<code>""" ... """</code>	teks banyak baris / dokumentasi	Python
<code>0x 0b 0o</code>	awalan heksa, biner, oktal	Python, JS, C
<code>1e3</code>	$1 \times 10^3 = 1000$	Python, JS

Karakter escape dalam teks

Tulisan	Hasil	Tulisan	Hasil
<code>\n</code>	baris baru	<code>\\</code>	satu backslash
<code>\t</code>	tab (jarak lebar)	<code>\"</code>	tanda kutip dua
<code>\r</code>	kembali ke awal baris (Windows: <code>\r\n</code>)	<code>\'</code>	tanda kutip satu
<code>`n</code> (PowerShell)	baris baru	<code>`"</code> (PowerShell)	tanda kutip dua
<code>\u00e9</code>	karakter unicode (huruf e beraksen)	<code>\x41</code>	karakter kode heksa 41 = A

Lampiran C. Regex (pola pencarian teks) ringkas

Regex (*regular expression*) adalah bahasa pola untuk mencari teks. Dipakai di `-match` PowerShell, `re` Python, `grep`, dan pencarian VS Code (aktifkan tombol `. *`).

Pola	Arti	Cocok dengan
<code>.</code>	satu karakter apa saja	<code>a.c</code> -> abc, a-c
<code>\d \D</code>	angka / bukan angka	<code>\d\d</code> -> 42
<code>\w \W</code>	huruf, angka, garis bawah / lawannya	<code>\w+</code> -> kata_1
<code>\s \S</code>	spasi/tab/enter / bukan	
<code>^ \$</code>	awal baris / akhir baris	<code>^ERROR</code> -> baris berawalan ERROR
<code>* + ?</code>	nol atau lebih / satu atau lebih / nol atau satu	<code>ab*</code> -> a, ab, abbb
<code>{n} {n, m}</code>	tepat n kali / n sampai m kali	<code>\d{4}</code> -> 2026
<code>[abc] [a-z] [^abc]</code>	salah satu karakter / rentang / bukan	<code>[A-Z][a-z]+</code> -> Budi
<code>()</code>	kelompok (bisa diambil)	<code>(\d+)-(\d+)</code>
<code>a b</code>	a atau b	<code>error fatal</code>
<code>\b</code>	batas kata	<code>\bkucing\b</code>
<code>\.</code>	titik harfiah (escape)	<code>file\.</code> txt

Lampiran D. Karakter khusus di URL

Bagian URL	Contoh	Arti
<code>https://</code>	protokol	<code>http</code> biasa, <code>https</code> terenkripsi
<code>host</code>	<code>localhost, example.com</code>	alamat komputer tujuan
<code>:port</code>	<code>:8081</code>	pintu program
<code>/jalur</code>	<code>/api/siswa/5</code>	bagian yang diminta
<code>?</code>	<code>?kelas=10A&hadir=true</code>	awal parameter pencarian
<code>&</code>	<code>kelas=10A&hadir=true</code>	pemisah antar parameter
<code>=</code>	<code>kelas=10A</code>	pasangan nama = nilai
<code>#</code>	<code>halaman#bab2</code>	bagian halaman (tidak dikirim ke server)
<code>%20</code>	<code>nama%20siswa</code>	spasi; <code>%</code> + kode heksa untuk karakter khusus

Lampiran E. Shortcut VS Code yang berguna

Tombol	Fungsi	Tombol	Fungsi
Ctrl+S	simpan	Ctrl+Z / Ctrl+Y	urungkan / ulangi
Ctrl+P	buka file cepat (ketik nama)	Ctrl+Shift+P	palet perintah (semua fitur)
Ctrl+`	buka/tutup terminal	Ctrl+B	buka/tutup panel samping
Ctrl+/	beri/hapus komentar baris	Alt+Up / Down	geser baris
Ctrl+D	pilih kata sama berikutnya	Ctrl+F / Ctrl+H	cari / ganti
Shift+Alt+F	rapikan format kode	F5	jalankan/debug
Ctrl+Shift+F	cari di seluruh proyek	Ctrl+Shift+M	panel Problems (daftar error)
Ctrl+Klik nama fungsi	loncat ke definisinya	F2	ganti nama variabel di semua tempat

Penutup

Tidak ada orang yang hafal semuanya. Programmer berpengalaman pun sering membuka dokumentasi. Yang dilatih adalah **mengenali pola** dan **tahu cara mencari**: baca pesan error, gunakan **Get-Help** / **--help**, dan cari kata kunci yang tepat.